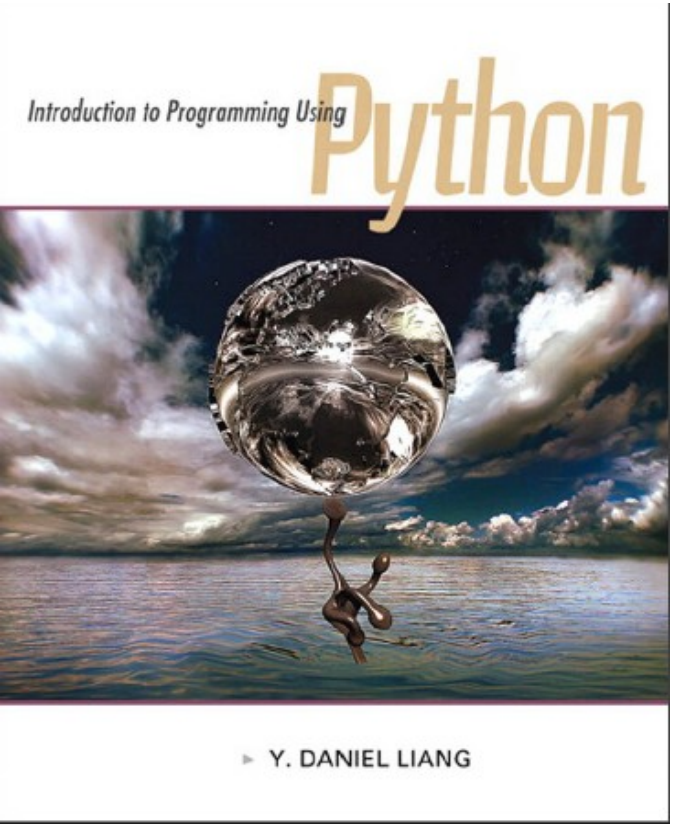
Python Programming

Chapter 06 Functions



Prepared By:

Eng. Eyad J. R. Elfarra

6.1 Introduction

Functions can be used to define reusable code and organize and simplify code.

Opening Problem

- Find the sum of integers from 1 to 10, from 20 to 37, and
- from 35 to 49, respectively.

Problem

```
sum = 0
for i in range(1, 10):
    sum += i
print("Sum from 1 to 10 is", sum)

sum = 0
for i in range(20, 37):
    sum += i
print("Sum from 20 to 37 is", sum)

sum = 0
for i in range(35, 49):
    sum += i
print("Sum from 35 to 49 is", sum)
```

Problem

```
sum = 0
for i in range(1, 10):
    sum += i
print("Sum from 1 to 10 is", sum)

sum = 0
for i in range(20, 37):
    sum += i
print("Sum from 20 to 37 is", sum)

sum = 0
for i in range(35, 49):
    sum += i
print("Sum from 35 to 49 is", sum)
```

Solution

```
def sum(i1, i2):
    result = 0
    for i in range(i1, i2):
        result += i
    return result

def main():
    print("Sum from 1 to 10 is", sum(1, 10))
    print("Sum from 20 to 37 is", sum(20, 37))
    print("Sum from 35 to 49 is", sum(35, 49))

main() # Call the main function
```

Objectives

- To define functions (§ 6.2).
- To invoke value returning functions (§ 6.3).
- To invoke functions that does not return a value (§ 6.4).
- To pass arguments by values (§ 6.5).
- To pass arguments by values (§ 6.6).
- To develop reusable code that is modular, easy to read, easy to debug, and easy to maintain (§ 6.7).
- To create modules for reusing functions (§§ 6.7 6.8).
- To determine the scope of variables (§ 6.9).
- To define functions with default arguments (§ 6.10).
- To return multiple values from a function (§ 6.11).
- To apply the concept of function abstraction in software development (§ 6.12).
- To design and implement functions using stepwise refinement § 6.13).

6.2 Defining a Function

A function definition consists of the function's name, parameters, and body.

Python Functions

There are **two** kinds of functions in Python.

- **Built-in functions** that are provided as part of Python - print(), input(), type(), float(), int() ...

- **Functions that we define ourselves** and then use

We treat the built-in function names as “new” **reserved words** (i.e., we avoid them as variable names)

Function Definition

In Python a **function** is some *reusable* code that takes **arguments**(s) as input, does some computation, and then returns a result or results

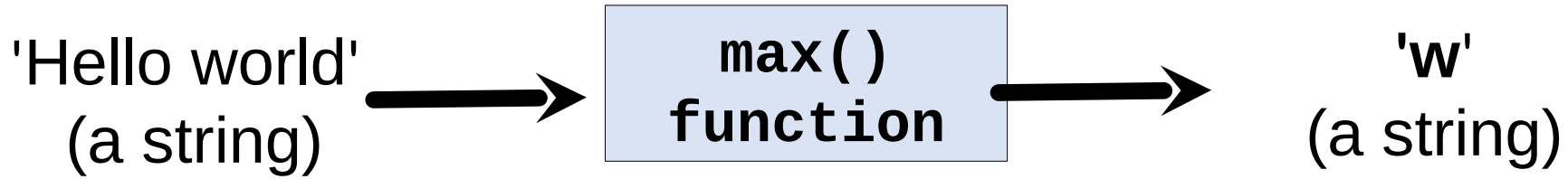
We define a **function** using the **def** reserved word

We call/invoke the **function** by using the function name, parentheses, and **arguments** in an expression

Max Function

```
>>> big = max('Hello world')
>>> print(big)
w
```

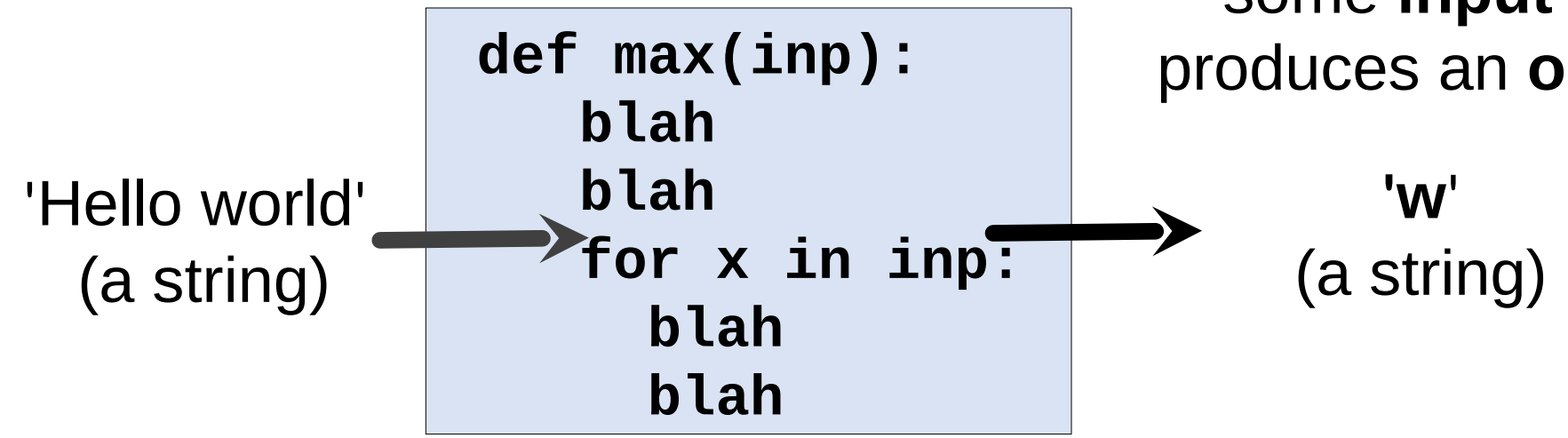
A **function** is **some stored code** that we use. A function takes some **input** and produces an **output**.



Max Function

```
>>> big = max('Hello world')
>>> print(big)
w
```

A **function** is **some stored code** that we use. A function takes some **input** and produces an **output**.



Building our Own Functions

We create a new function using the **def** keyword followed by optional parameters in parentheses

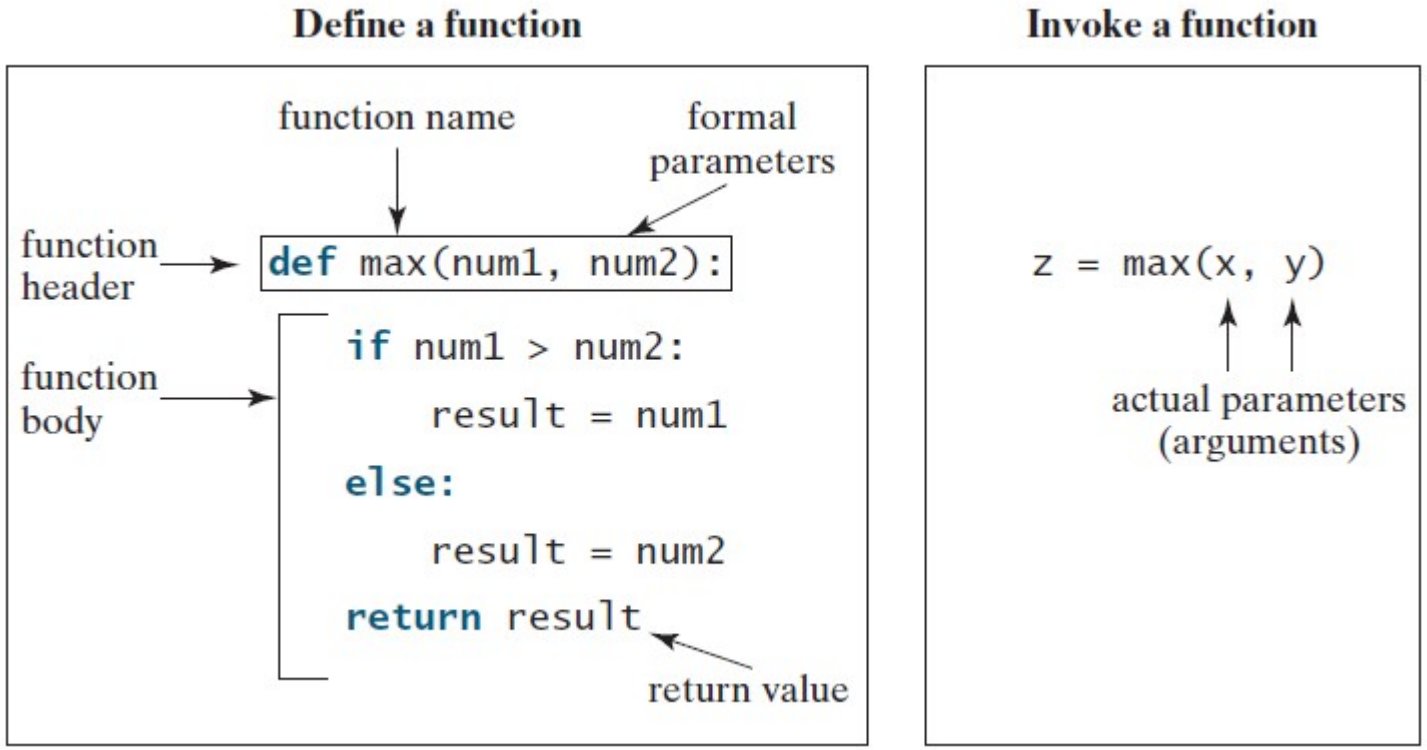
We **indent** the body of the function

This **defines** the function but **does not** execute the body of the function

```
def print_lyrics():
    print("I'm a Programmer, and I'm okay.")
    print('I work all night and all day.')
```

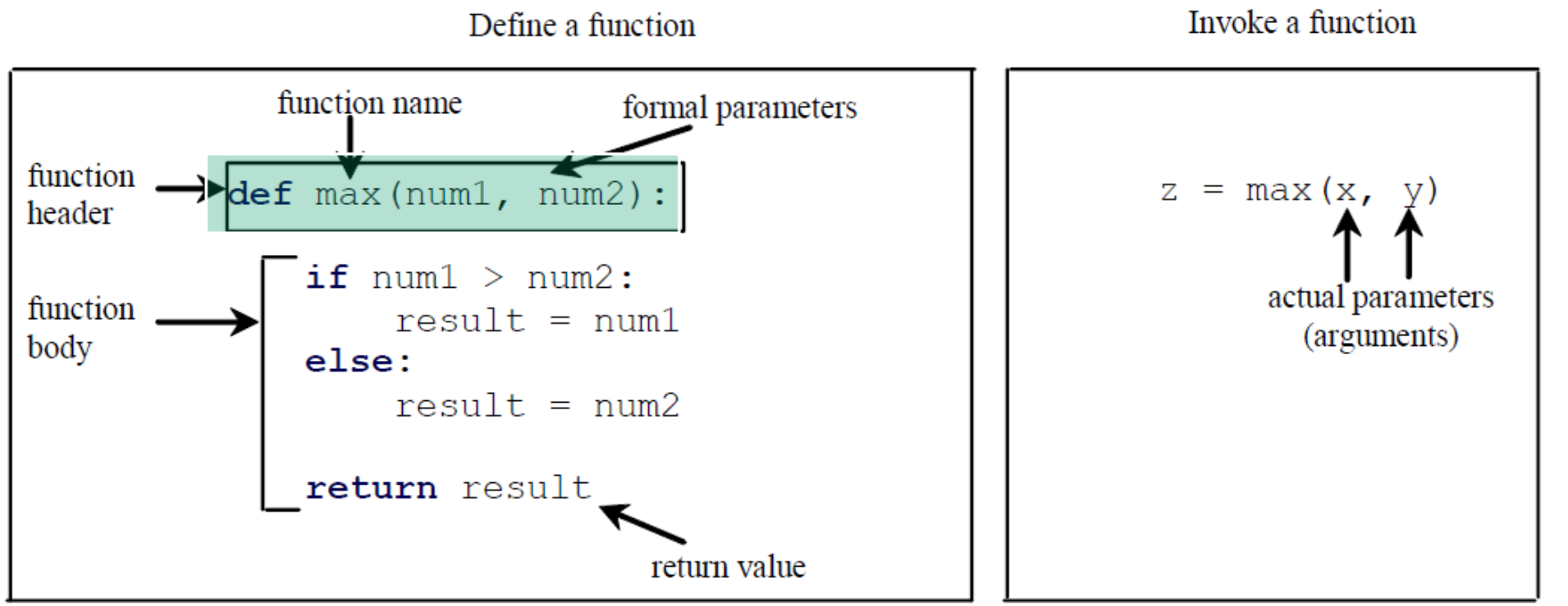
Defining Functions

- A function is a collection of statements that are grouped together to perform an operation.



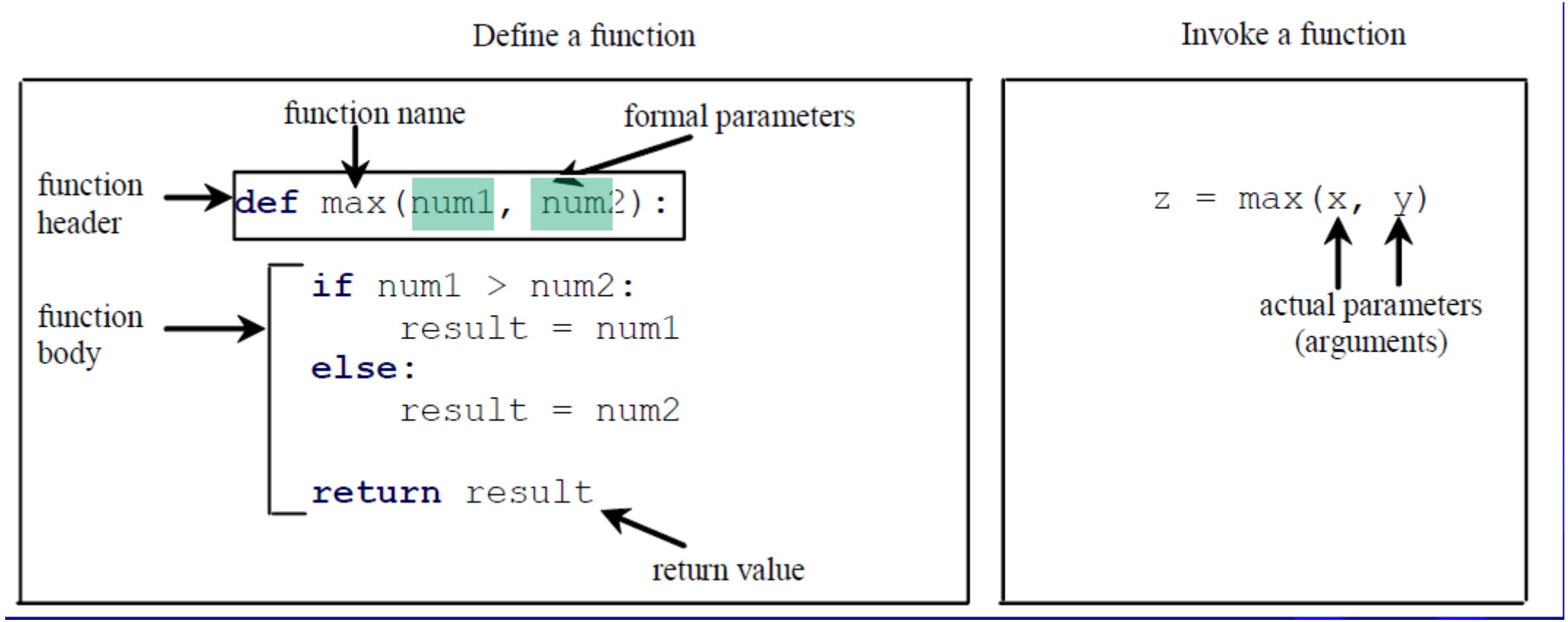
Function Header

- A function contains a header and body.
- The **header** begins with the **def** *keyword*, followed by **function's name** and **parameters**, followed by a **colon**.



Formal Parameters

- The variables defined in the function header are known as *formal parameters*.



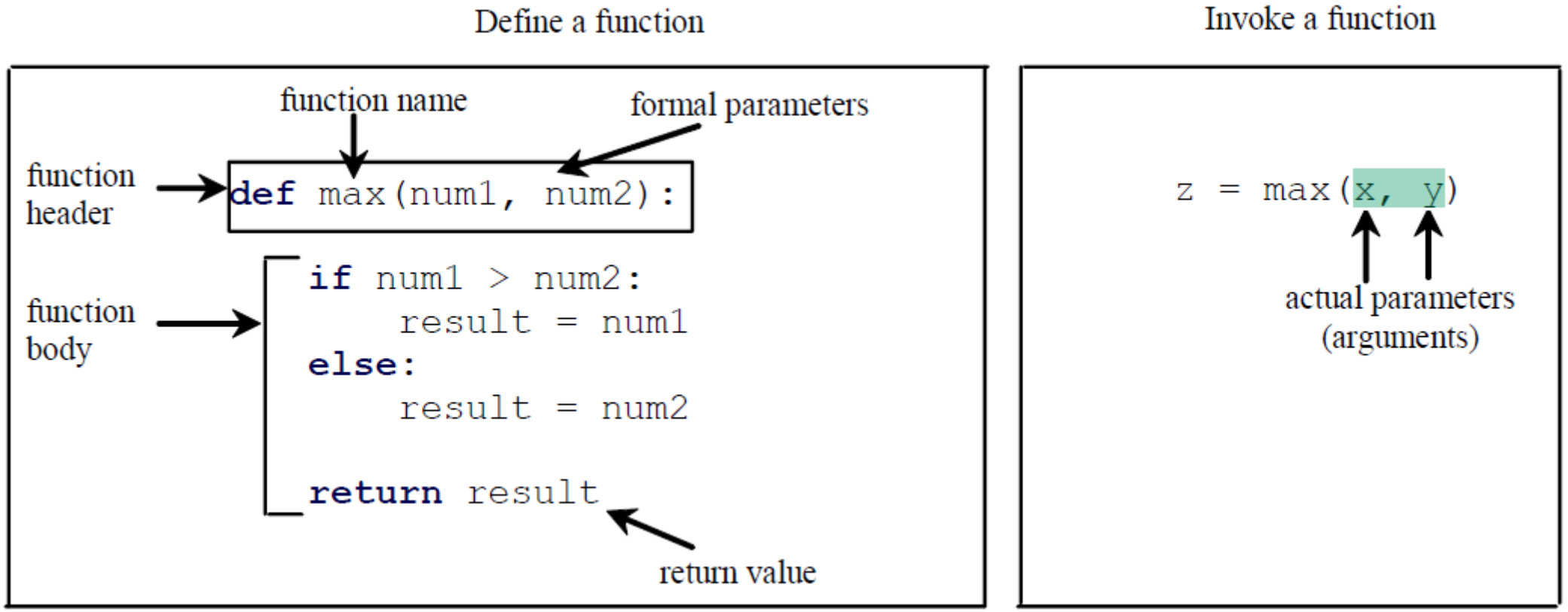
Formal Parameters (Parameters)

A **parameter** is a variable which we use **in** the function **definition**. It is a “handle” that allows the code in the **function** to access the **arguments** for a particular **function** invocation.

```
>>> def greet(lang):
...     if lang == 'es':
...         print('Hola')
...     elif lang == 'fr':
...         print('Bonjour')
...     else:
...         print('Hello')
...
>>> greet('en')
Hello
>>> greet('es')
Hola
>>> greet('fr')
Bonjour
>>>
```

Actual Parameters

- When a function is invoked, you pass a value to the parameter.
- This value is referred to as **actual parameter** or **argument**.



Arguments

An **argument** is a value we pass into the **function** as its **input** when we call the function

We use **arguments** so we can direct the **function** to do different kinds of work when we call it at **different** times

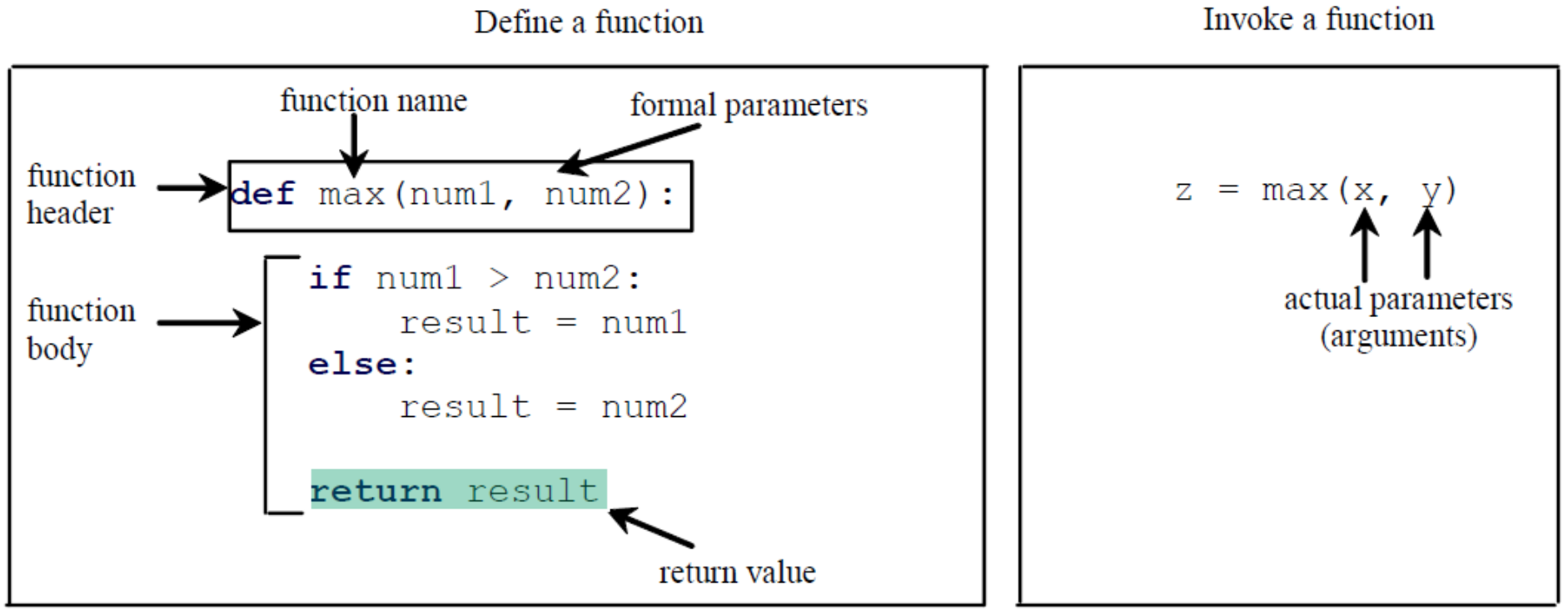
We put the **arguments** in parentheses after the **name** of the function

```
big = max('Hello world')
```



Return Value

- Often a function will take its arguments, do some computation, and return a value to be used as the value of the function call in the calling expression.
- A function *may* return a value using the **return** keyword.



Multiple Parameters / Arguments

We can define more than one **parameter** in the **function definition**

We simply add more **arguments** when we call the **function**

We match the number and order of arguments and parameters

```
def addtwo(a, b):
    added = a + b
    return added

x = addtwo(3, 5)
print(x)

8
```

Definitions and Uses

Once we have **defined** a function, we can **call** (or **invoke**) it as many times as we like

This is the **store** and **reuse** pattern

6.3 Calling a Function

Calling a function executes the code in the function.

Calling a Function

- Calling a function executes the code in the function.
- The program that calls the function is called the **caller**.
- Functions can be called in two ways:
  - Value-returning function
  - Non-value-returning function

Value-Returning Function

```
larger = max(3, 4)
```

- Returns a value.
- The returned value is assigned to larger.

Function Without Return Value

```
print("Programming is fun!")
```

- print() does not return a value.
- The function call must be a statement.

Note

- A value-returning function can also be called as a statement.
- In this case, the **return** value is **ignored**.

```
max(3, 4)
```

Program Control

- Calling a function transfers control to the function.
- The function executes its statements.
- Control returns to the caller when:
  - return executes, or
  - the function finishes.

TestMax.py

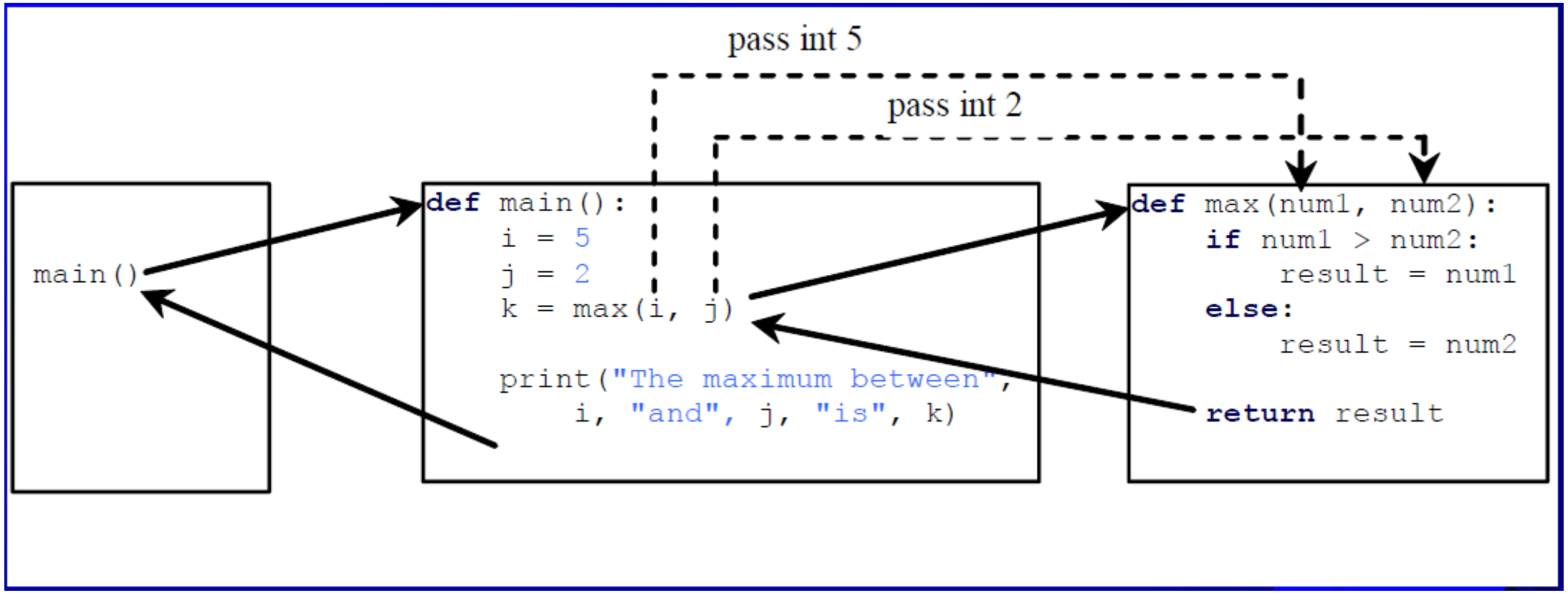
```
The larger number of 5 and 2 is 5
```

```
# Return the max of two numbers
def max(num1, num2):
    if num1 > num2:
        result = num1
    else:
        result = num2
    return result

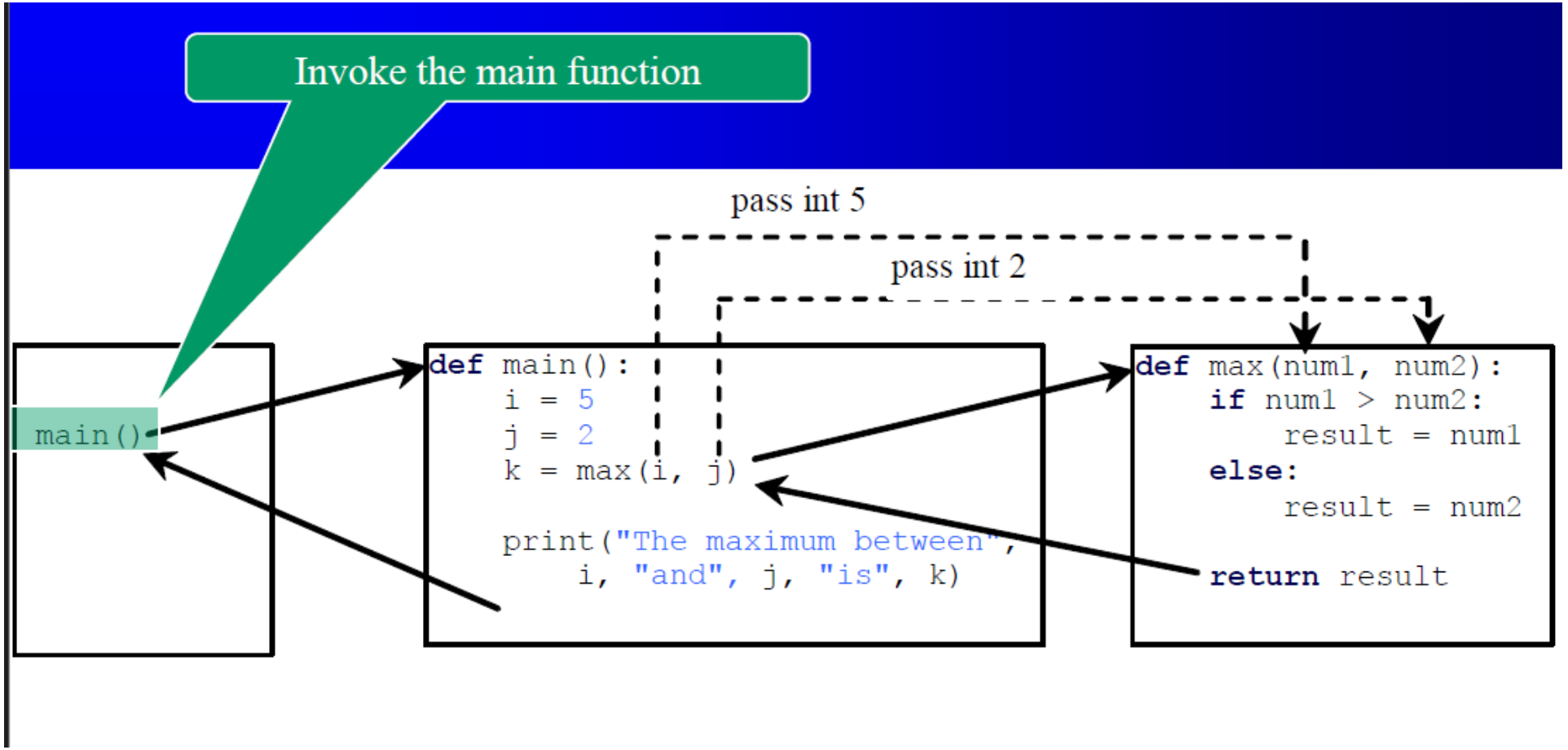
def main():
    i = 5
    j = 2
    k = max(i, j)  # Call the max function
    print("The larger number of", i, "and", j, "is", k)

main()  # Call the main function
```

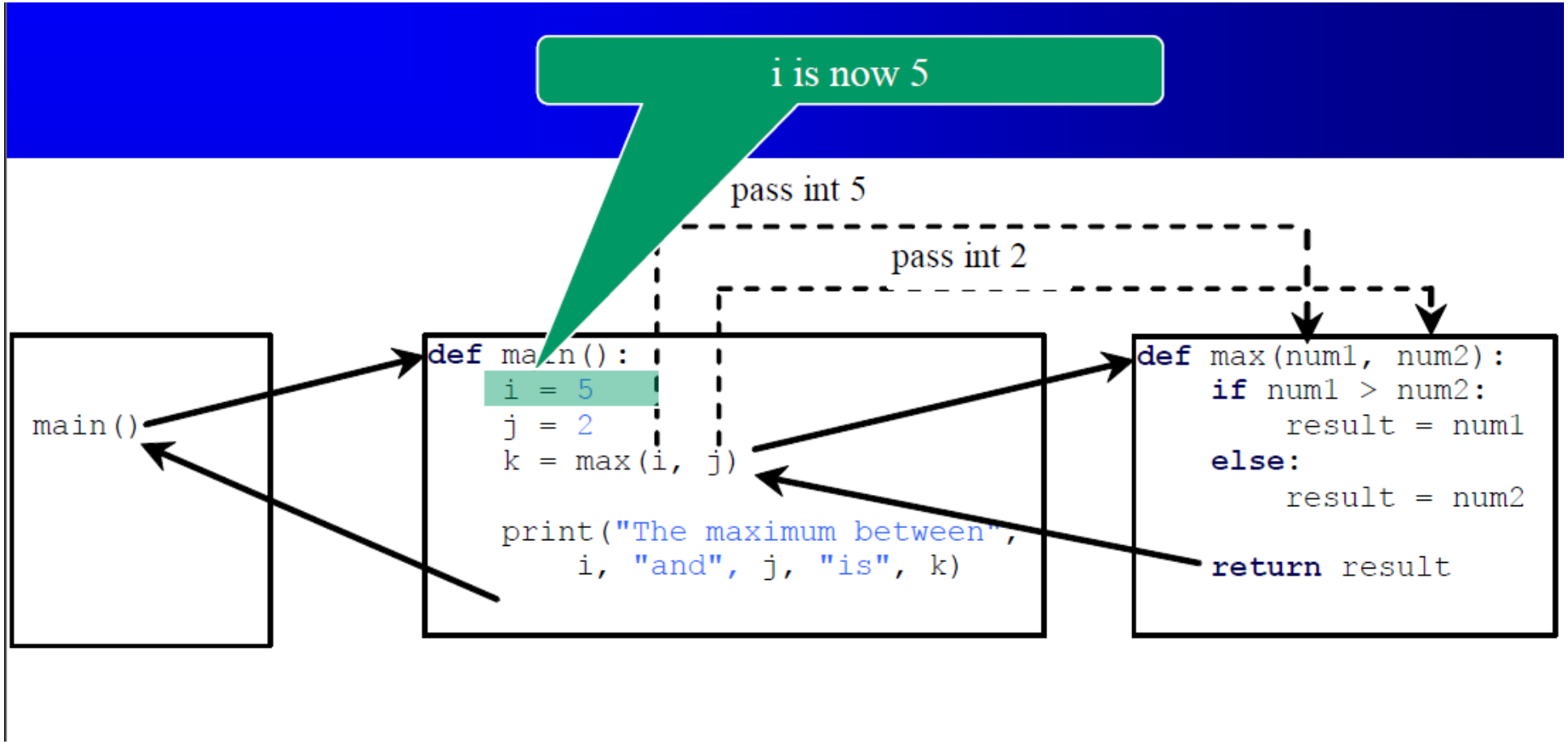
Calling Functions, cont.



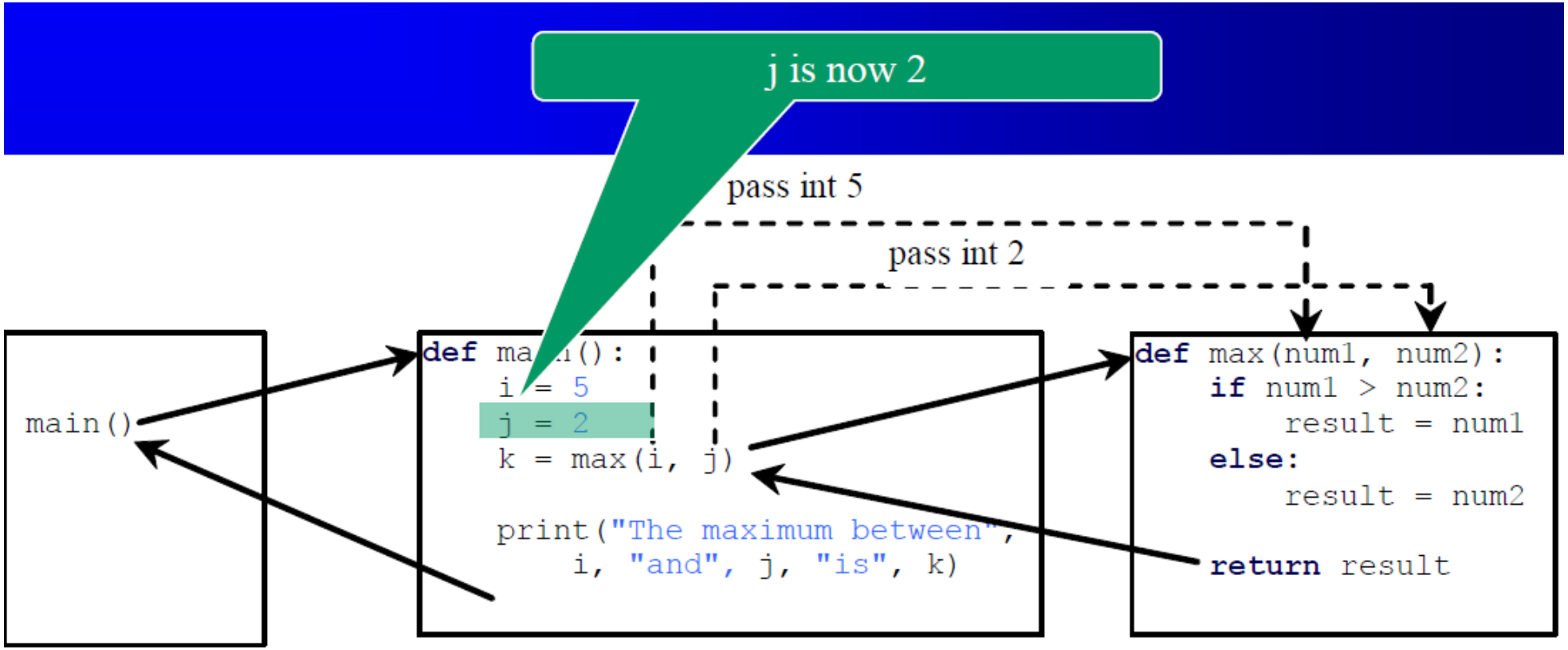
Trace Function Invocation



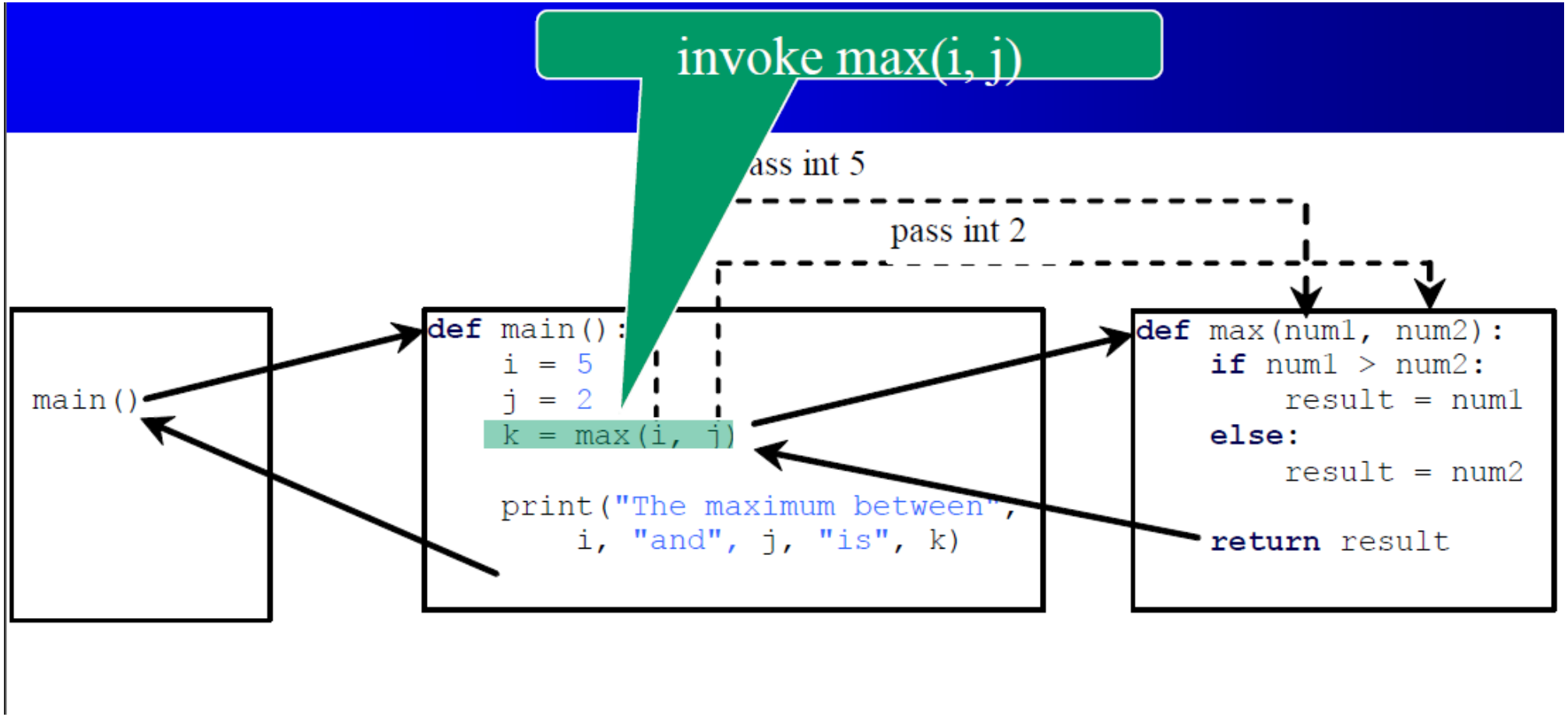
Trace Function Invocation



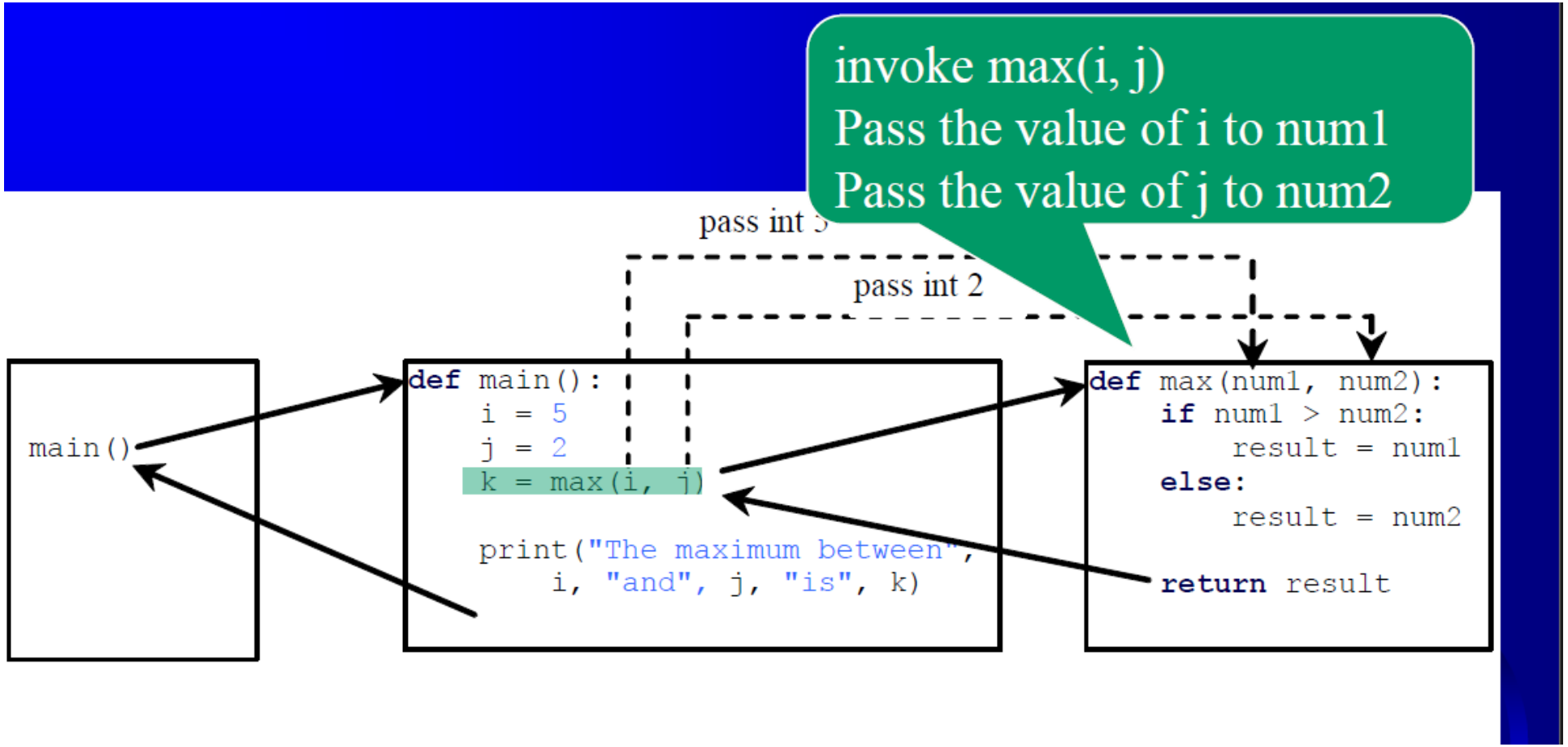
Trace Function Invocation



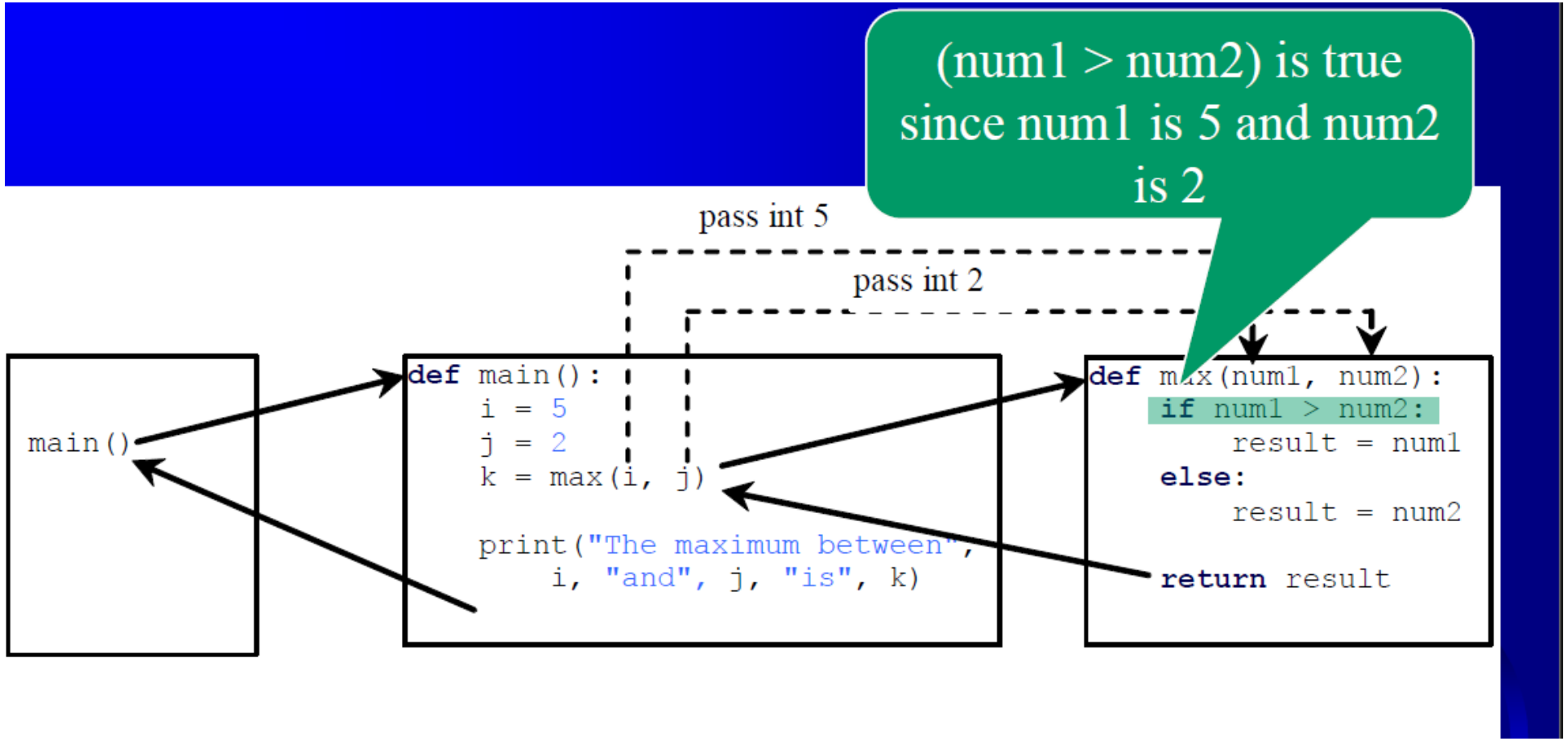
Trace Function Invocation



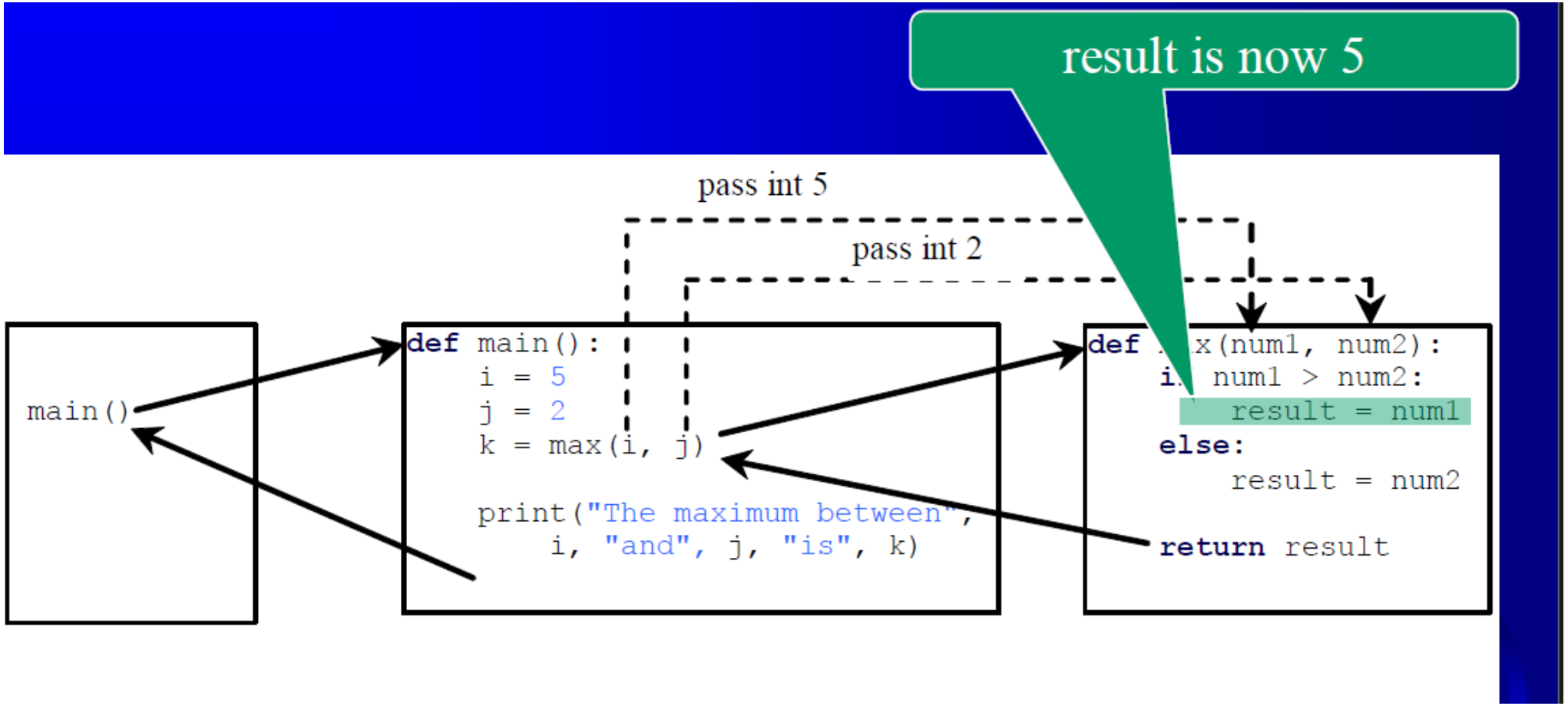
Trace Function Invocation



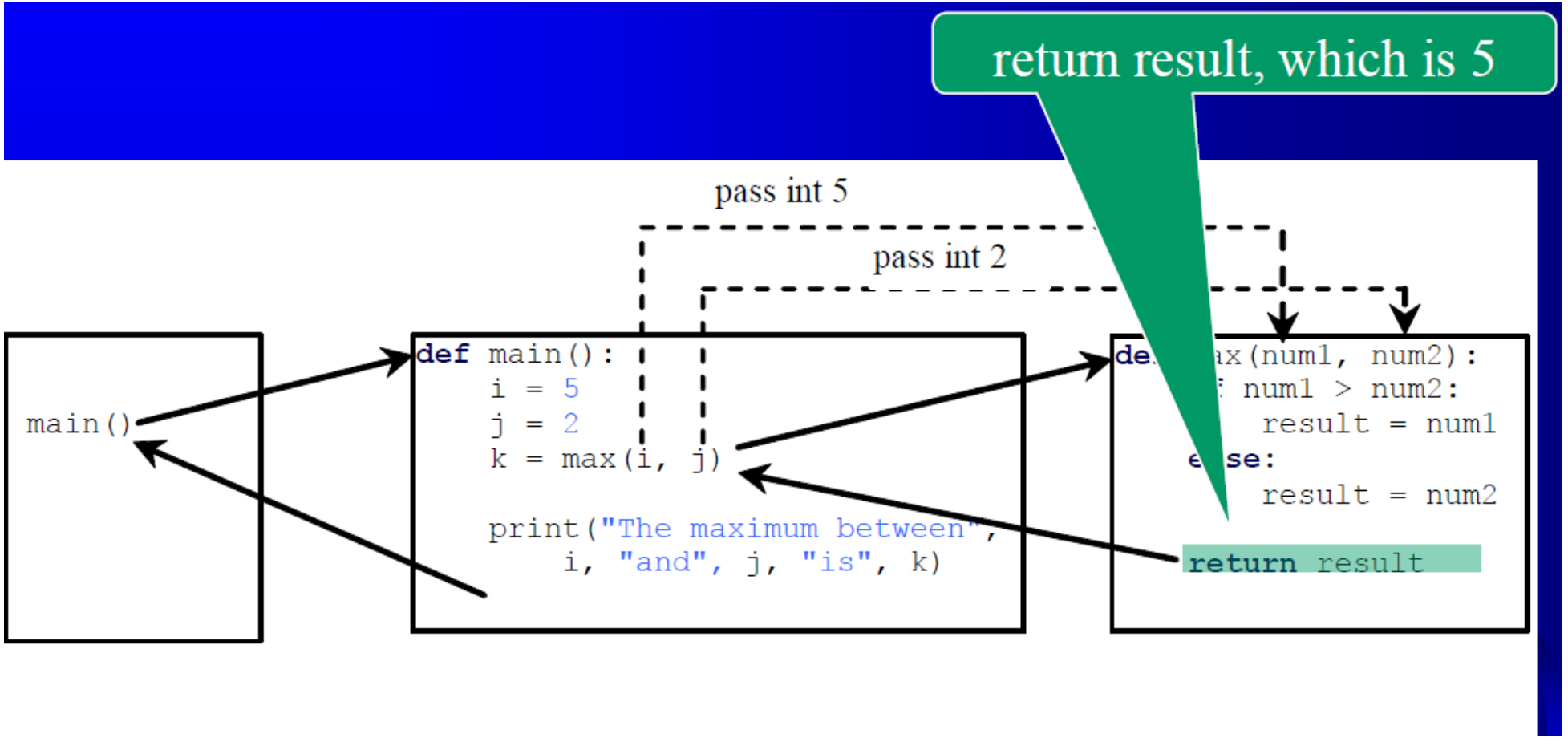
Trace Function Invocation



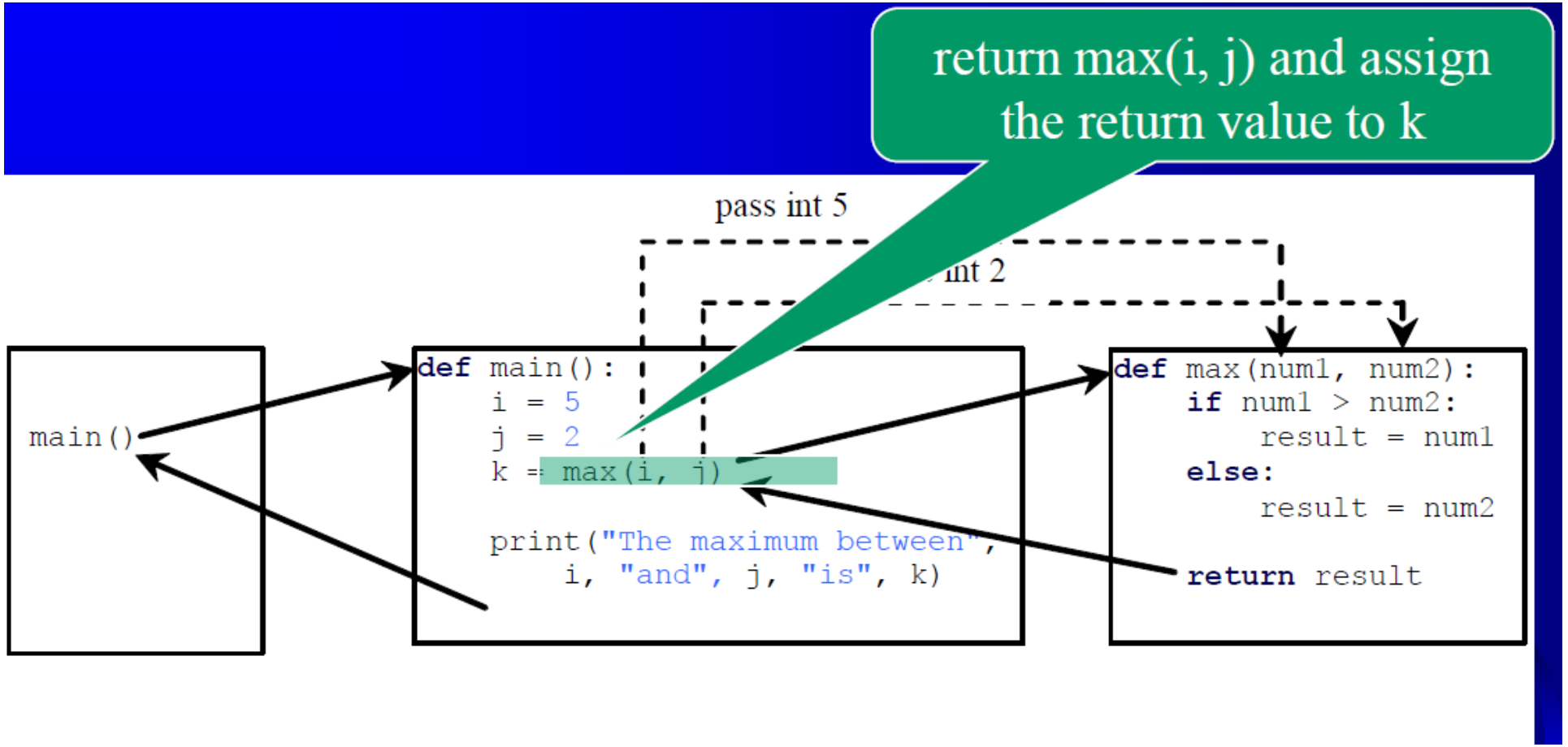
Trace Function Invocation



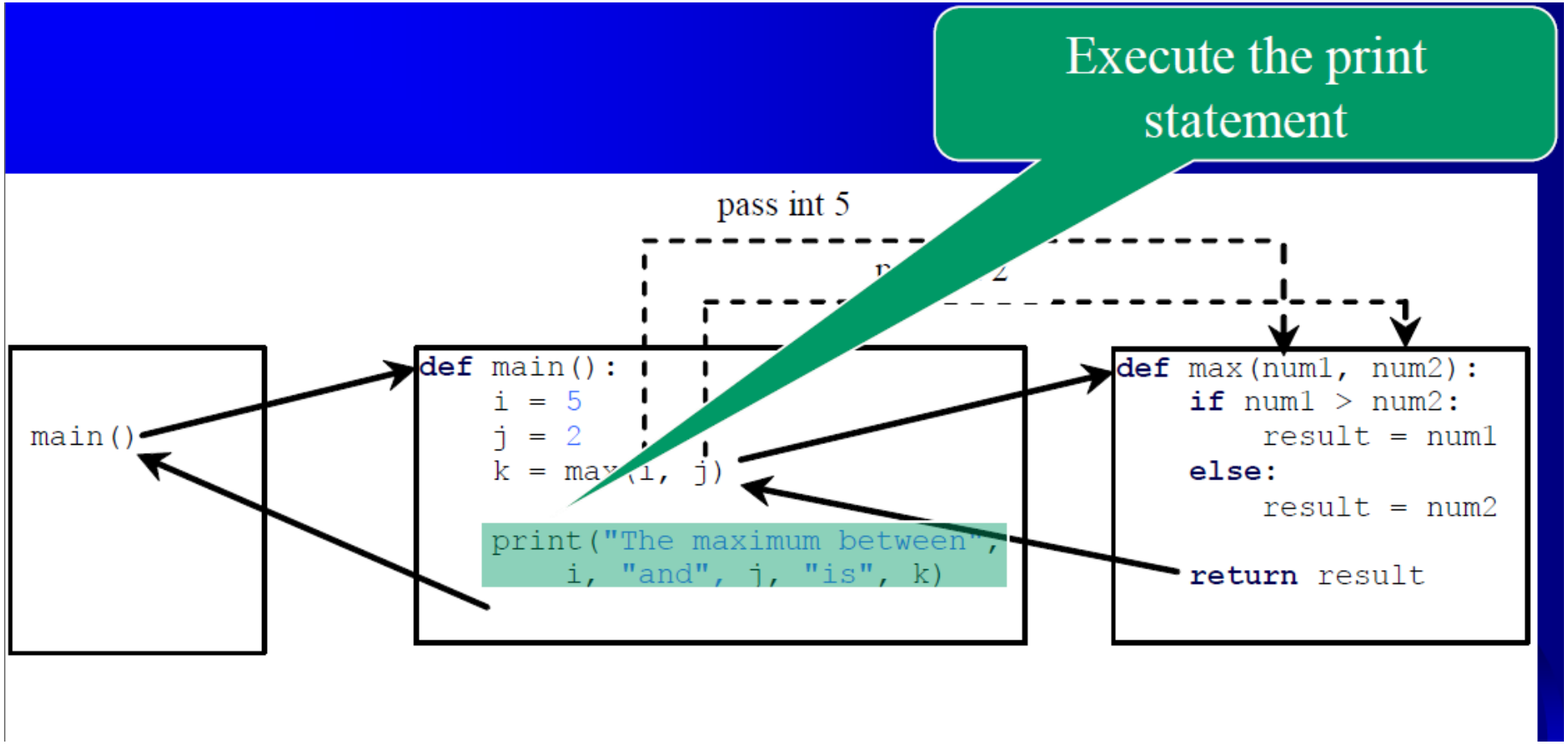
Trace Function Invocation



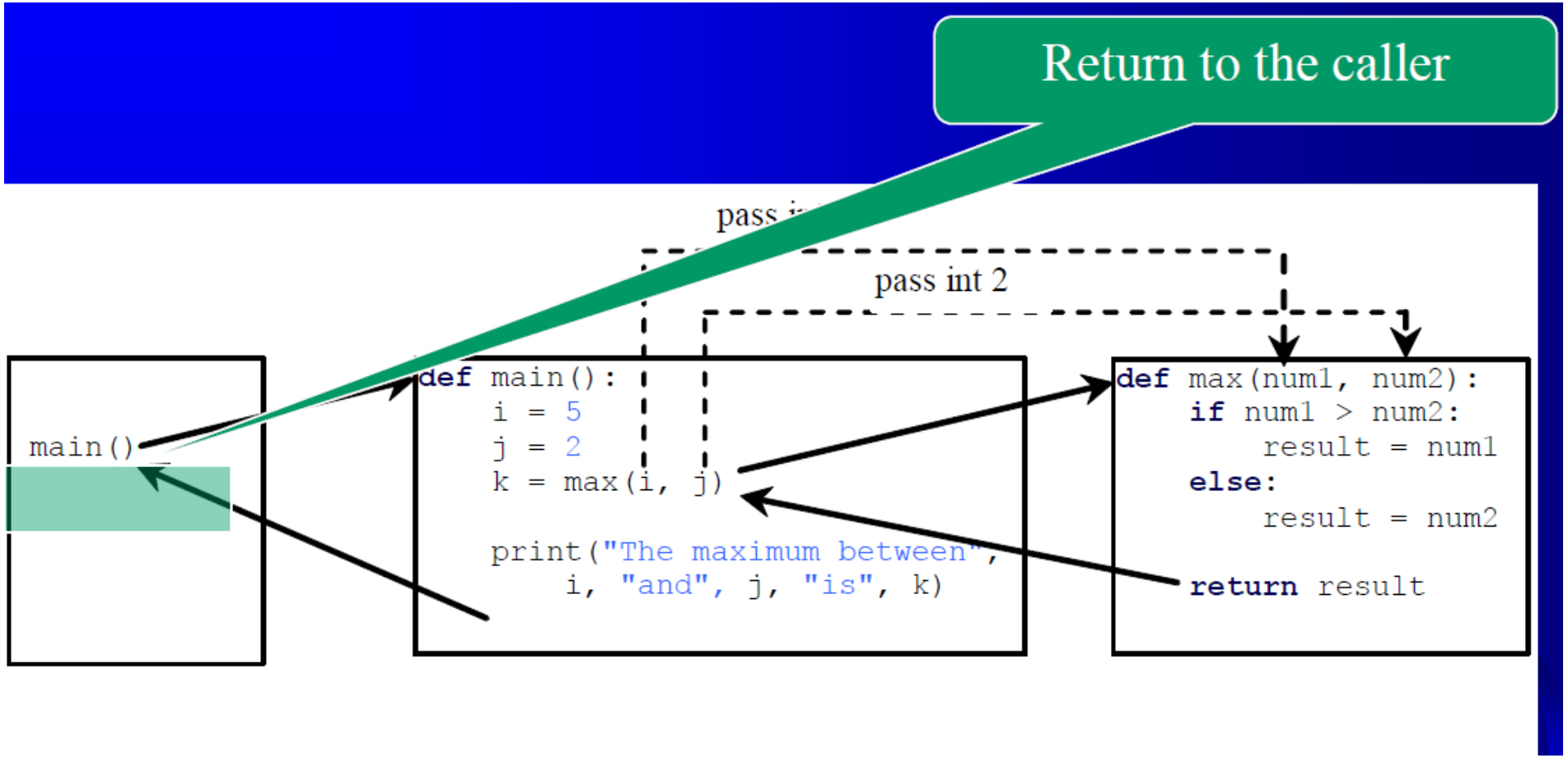
Trace Function Invocation



Trace Function Invocation



Trace Function Invocation



Note

- Here main is defined after max.
- In Python, functions can be defined in any order in a script file as long as the function is in the memory when it is called.
- You can also define main before max.

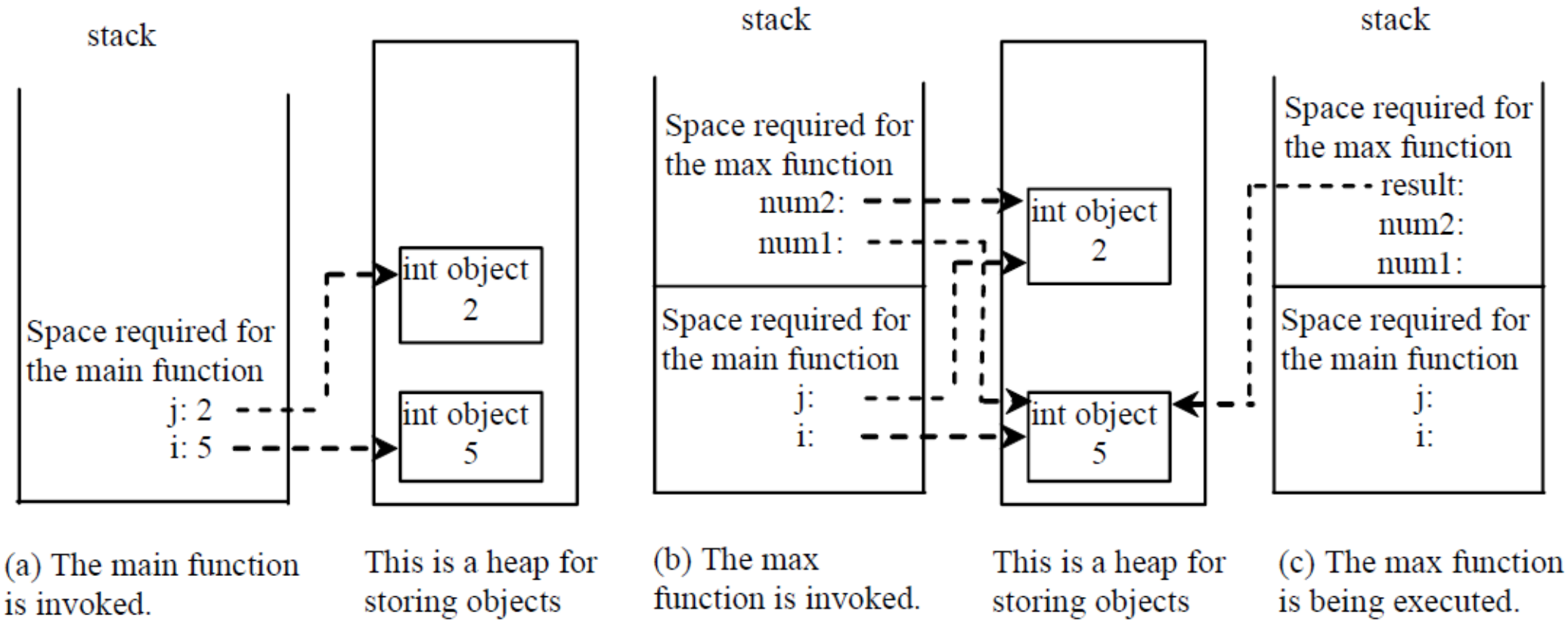
Activation Record

- Each time a function is invoked:
  - The system creates an **activation record**.
- The activation record stores:
  - Function arguments
  - Local variables
- **Call Stack**
  - Activation records are placed in memory called the **call stack**.
  - Other names:
    - Execution Stack
    - Runtime Stack
    - Machine Stack
    - Stack

Function Calls

- **Function Calls**
  - When one function calls another:
    - The caller’s activation record remains in the stack.
    - A new activation record is created for the called function.
- **Returning from a Function**
  - When a function finishes:
    - Control returns to the caller.
    - Its activation record is removed from the stack.
- **Last-In, First-Out (LIFO)**
  - A call stack works in **Last-In, First-Out** order.
  - Meaning:
    - The last function called
    - Is the first function removed
- Python creates and stores objects in a separate memory space called *heap*.

Call Stacks



(a) The main function is invoked.

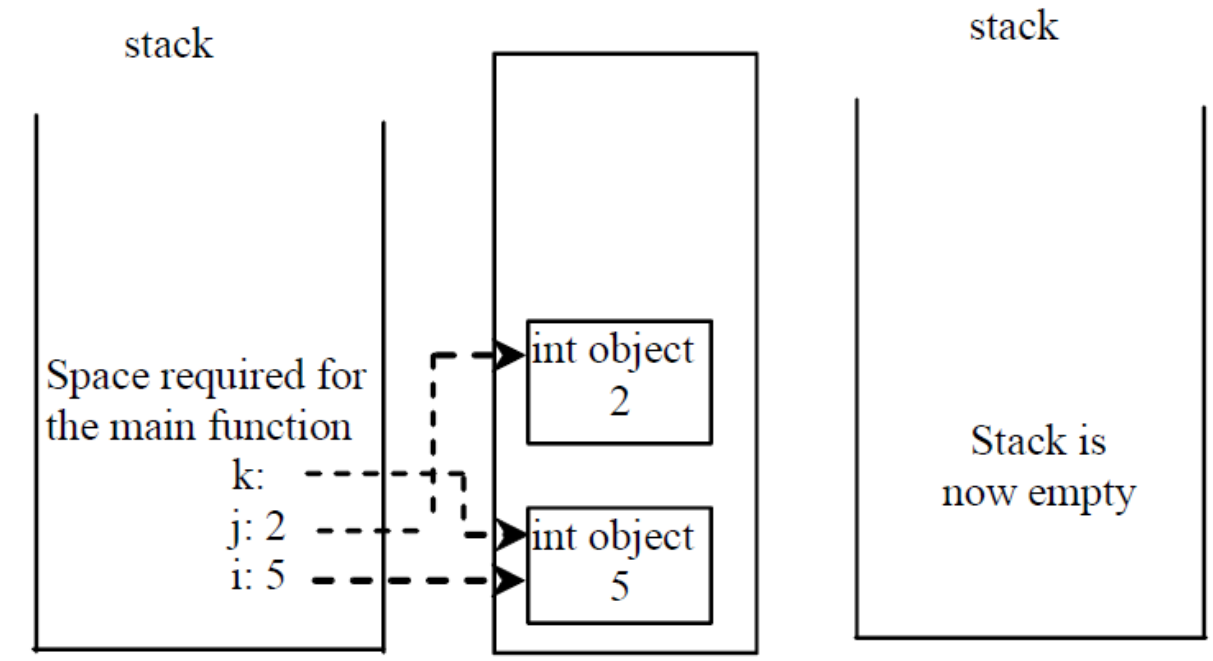
This is a heap for storing objects

(b) The max function is invoked.

This is a heap for storing objects

(c) The max function is being executed.

Call Stacks



(d) The max function is finished and the return value is sent to k.

This is a heap for storing objects

(e) The main function is finished.

Program Memory

- During execution:
  - Functions use the **stack**
  - Objects are stored in the **heap**
- After main() **finishes**:
  - The **stack** becomes **empty**.
- **Automatic Memory Management**
  - Python automatically destroys unused objects.
  - This process is handled by the Python interpreter.
  - This is known as **automatic garbage collection**.

6.4 Functions with/without Return Values

A function does not have to return a value.

Return Value

A “fruitful” **function** is one that produces a **result** (or **return value**)

The **return** statement ends the **function** execution and “sends back” the **result** of the **function**

```
>>> def greet(lang):
...     if lang == 'es':
...         return 'Hola'
...     elif lang == 'fr':
...         return 'Bonjour'
...     else:
...         return 'Hello'
...
>>> print(greet('en'),'Glenn')
Hello Glenn
>>> print(greet('es'),'Sally')
Hola Sally
>>>
print(greet('fr'),'Michael')
Bonjour Michael
>>>
```

Void (non-fruitful) Functions

When a function does not return a value, we call it a “**void**” function

Functions that return values are “fruitful” functions

Void functions are “not fruitful”

The None Value

- A function that does not return a value is known as a **void** function in other programming languages such as Python, C++, and C#.
- In Python, such function returns a special **None**.

```
def sum(number1, number2):
    total = number1 + number2
print(sum(1, 2))
```

PrintGradeFunction.py

```
Enter a score: 78.5 ↵Enter
The grade is C
```

```
def printGrade(score):
    if score >= 90.0:
        print('A')
    elif score >= 80.0:
        print('B')
    elif score >= 70.0:
        print('C')
    elif score >= 60.0:
        print('D')
    else:
        print('F')

def main():
    score = eval(input("Enter a score: "))
    print("The grade is", end=" ")
    printGrade(score)

main()
```

ReturnGradeFunction.py

```
Enter a score: 78.5 ↵Enter
The grade is C
```

```
def getGrade(score):
    if score >= 90.0:
        return 'A'
    elif score >= 80.0:
        return 'B'
    elif score >= 70.0:
        return 'C'
    elif score >= 60.0:
        return 'D'
    else:
        return 'F'

def main():
    score = eval(input("Enter a score: "))
    print("The grade is", getGrade(score))

main()
```

Note(1)

- In Python, **every function returns a value**
- Even if you do not write a return statement
- The returned value can be used or ignored
- Functions Without return
  - If a function has no return statement:
    - Python automatically returns a special value: **None**
  - Such functions are called:
    - **None functions**

What is None?

- None means:
  - No value
  - No object reference
- It is often used to indicate:
  - “nothing returned”
  - “empty result”

```
def sum(number1, number2):
        total = number1 + number2

print(sum(1, 2))
```

```
...
>>> print(sum(1, 2))
None
>>>
```

Note(2)

- A return statement is not needed for a None function, but it can be used for terminating the function and returning control to the function's caller.
- The syntax is simply:

```
return
```

or

```
return None
```

- This is rarely used, but it is sometimes useful for circumventing the normal flow of control in a function that does not return any value. For example, the following code has a return statement to terminate the function when the score is invalid.

Note(2)

```
Enter a score: 105
The grade is Invalid score
```

```
Enter a score: 95
The grade is A
done::::::::::::::::::::::::::::::
```

PrintGradeWithTermination.py

```python
def printGrade(score):
    if score < 0 or score > 100:
        print("Invalid score")
        return # Same as return None
    if score >= 90.0:
        print('A')
    elif score >= 80.0:
        print('B')
    elif score >= 70.0:
        print('C')
    elif score >= 60.0:
        print('D')
    else:
        print('F')

    print("done::::::::::::::::::::::::::::::")

def main():
    score = eval(input("Enter a score: "))
    print("The grade is", end=" ")
    printGrade(score)

main()
```

6.5 Positional and Keyword Arguments

A function's arguments can be passed as positional arguments or keyword arguments.

Arguments

- The power of a function is its ability to work with parameters.
- There are **two** kinds of arguments:
  - Positional arguments
  - Keyword arguments.

Passing Arguments by Positions

- Suppose you invoke the function using
  - nPrintln(“Welcome to Python”, 5)
  - What is the output?
- Suppose you invoke the function using
  - nPrintln(“Computer Science”, 15)
  - What is the output?
  - What is wrong
    - nPrintln(4, “Computer Science”)

```
def nPrintln(message, n):
    for i in range(0, n):
        print(message)
```

Keyword Arguments

- What is wrong
  - nPrintln(4, “Computer Science”)
- Is this OK?
  - nPrintln(n = 4, message = “Computer Science”)

Mixing Positional and Keyword Arguments

- In Python, you can mix positional and keyword arguments in function calls.
- However, there **is an important rule** about **their order**.

```
def f(p1, p2, p3):
    print(p1,p2,p3)
```

- Valid Function Call
  - f(30, p2 = 4, p3 = 10)
  - **30** is a positional argument (assigned to p1)
  - **p2 = 4** and **p3 = 10** are keyword arguments
  - This is **correct usage**

```
>>> f(30, p2 = 4, p3 = 10)
...
    30 4 10
```

Mixing Positional and Keyword Arguments

- Invalid Function Call
  - f(30, p2 = 4, 10)
  - Incorrect
  - A positional argument (**10**) appears after a keyword argument (**p2 = 4**)

```
>>> f(30, p2 = 4, 10)
SyntaxError: positional argument follows keyword argument
>>>
```

- Rules
  - Positional arguments **must come before** keyword arguments
  - **Once you use a keyword** argument,
    - all following arguments must also be keyword arguments

6.6 Passing Arguments by Reference Values

When you invoke a function with arguments, each argument's reference is passed by value to the parameter in the function.

Pass by Value

- In Python, all data are objects.
- A variable for an object is **actually a reference** to the object.
- When you invoke a function with a parameter:
  - the **reference value** of the argument is **passed** to the parameter.
  - This is referred to as **pass-by-value**.
  - For simplicity, we say that the value of an argument is passed to a parameter when invoking a function.
  - Precisely, the value is actually a reference value to the object.
- If the argument is a number or a string, the argument is not affected,
  - regardless of the changes made to the parameter inside the function.

Increment.py

```
def main():
    x = 1
    print("Before the call, x is", x)
    increment(x)
    print("After the call, x is", x)

def increment(n):
    print("\tn inside the function is", n)
    n += 1

main()  # Call the main function
```

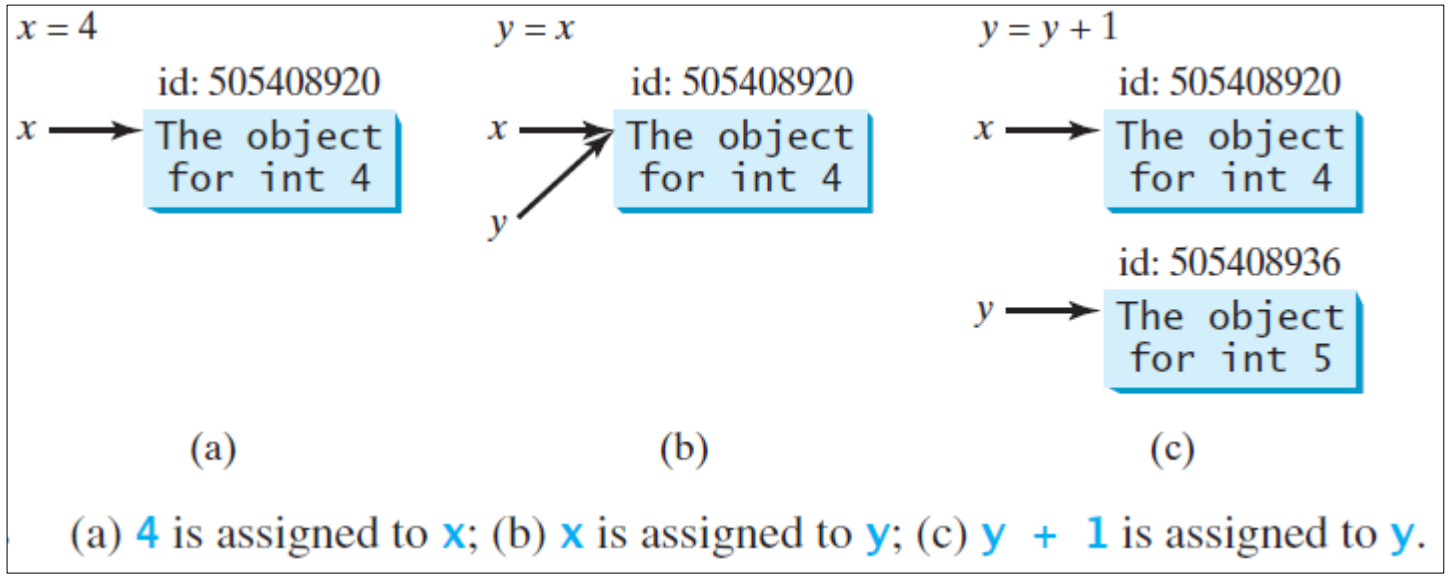
```
Before the call, x is 1
        n inside the function is 2
After the call, x is 1
```

Immutable Objects

- The parameter **n** is incremented by 1 in the function, but x is not changed no matter what the function does.
- The reason is that numbers and strings are known as **immutable** objects.
- The contents of immutable objects cannot be changed.
- Whenever you assign a new number to a variable,
  - Python **creates** a new object for the new number
  - and **assigns** the reference of the new object to the variable.

```
>>> x=4
>>> y=x
>>> id(x)
    140713891951832
>>> id(y)
    140713891951832
>>> y=y+1
>>> id(y)
    140713891951864
>>> id(x)
    140713891951832
>>>
```

Immutable Objects



6.7 Modularizing Code

Modularizing makes code easy to maintain and debug, and enables the code to be reused.

Modularizing Code

- **Functions and Code Reuse**
  - Functions help reduce redundant (repeated) code
  - They allow code reuse in multiple parts of a program
  - Improve readability and maintainability
- **Functions and Modularization**
  - Functions help break a program into smaller parts
  - This process is called **modularization**
  - Each function handles a specific task
  - This improves program quality and organization

Modules

- **Python Modules**
  - A module is a file that contains Python functions
  - It is saved with the extension .py
  - Example: my_functions.py
- **Using Modules**
  - A module can be imported into other programs
  - This allows reuse of existing functions
  - The module file should be in the **same directory** as your program

Python Module

- **Multiple Functions in a Module**
  - A module can contain more than one function
  - Each function must have a unique name
  - This helps organize related functions together
- **Built-in Python Modules**
  - Python provides ready-made modules.
  - Examples:
    - turtle (graphics and drawing)
    - random (random number generation)
    - math (mathematical functions)
  - These modules can be imported directly in any program

Code Modularization Example

LISTING 6.5 GCDFunction.py

```
1  # Return the gcd of two integers
2  def gcd(n1, n2):
3      gcd = 1 # Initial gcd is 1
4      k = 2   # Possible gcd
5
6      while k <= n1 and k <= n2:
7          if n1 % k == 0 and n2 % k == 0:
8              gcd = k # Update gcd
9          k += 1
10
11     return gcd # Return gcd
```

LISTING 6.6 TestGCDFunction.py

```
1  from GCDFunction import gcd # Import the gcd function
2
3  # Prompt the user to enter two integers
4  n1 = eval(input("Enter the first integer: "))
5  n2 = eval(input("Enter the second integer: "))
6
7  print("The greatest common divisor for", n1,
8      "and", n2, "is", gcd(n1, n2))
```

```
Enter the first integer: 45 ↵Enter
Enter the second integer: 75 ↵Enter
The greatest common divisor for 45 and 75 is 15
```

Notes

- By encapsulating the code for obtaining the gcd in a function, this program has several advantages:
  - It **isolates** the problem for computing the gcd from the rest of the code in the program.
    - Thus, the logic becomes clear and the program is easier to read.
  - Any **errors** for computing the gcd are confined to the gcd function, which narrows the scope of debugging.
  - The gcd function now can be **reused** by other programs.
- What happens if you define two functions with the same name in a module?
  - There is **no syntax error** in this case
  - But the **latter function definition prevails**.

PrimeNumberFunction.py

- This program divides a large problem into two subproblems.
- As a result, the new program is easier to read and easier to debug.
- The two functions:
  - First function check if the number is prime or not

```
# Check whether number is prime
def isPrime(number):
```

  - The second function print the prime numbers

```
def printPrimeNumbers(numberOfPrimes):
```

- Program anatomy



PrimeNumberFunction.py

```
The first 50 prime numbers are

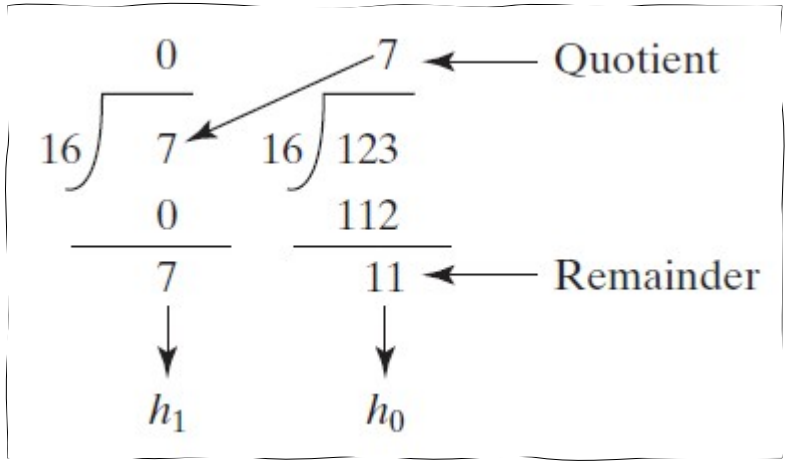
2    3    5    7    11   13   17   19   23   29
31   37   41   43   47   53   59   61   67   71
73   79   83   89   97   101  103  107  109  113
127  131  137  139  149  151  157  163  167  173
179  181  191  193  197  199  211  223  227  229
```

6.8 Case Study: Converting Decimals to Hexadecimals

This section presents a program that converts a decimal number to a hexadecimal number.

Problem: Converting Decimals to Hexadecimals

- The decimal number **123** is **7B** in hexadecimal. The conversion is done as shown below:



- To convert from Hexadecimal to decimal:

$$d = h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \ldots + h_2 \times 16^2 + h_1 \times 16^1 + h_0 \times 16^0$$

Hexadecimal (Base 16)	Decimal (Base 10)
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

Problem: Converting Decimals to Hexadecimals

- Write a function that converts a decimal integer to a hexadecimal.

```
Enter a decimal number: 1234  Enter
The hex number for decimal 1234 is 4D2
```

Problem: Converting Decimals to Hexadecimals

- Write a function that converts a decimal integer to a hexadecimal.

```
1  # Convert a decimal to a hex as a string
2  def decimalToHex(decimalValue):
3      hex = ""  # Hex string is initially empty
4
5      while decimalValue != 0:
6          hexValue = decimalValue % 16
7          hex = toHexChar(hexValue) + hex
8          decimalValue = decimalValue // 16
9
10     return hex
11
12 # Convert an integer to a single hex digit as a character
13 def toHexChar(hexValue):
14     if 0 <= hexValue <= 9:
15         return chr(hexValue + ord('0'))
16     else:  # 10 <= hexValue <= 15
17         return chr(hexValue - 10 + ord('A'))
18
19 def main():
20     # Prompt the user to enter a decimal integer
21     decimalValue = eval(input("Enter a decimal number: "))
22
23     print("The hex number for decimal",
24           decimalValue, "is", decimalToHex(decimalValue))
25
26 main()  # Call the main function
```

```
Enter a decimal number: 1234 Enter
The hex number for decimal 1234 is 4D2
```

	line#	decimalValue	hex	hexValue	toHexChar(hexValue)
	21	1234			
	3		""		
iteration 1	6			2	
	7		"2"		"2"
	8	77			
iteration 2	6			13	
	7		"D2"		"D"
	8	4			
iteration 3	6			4	
	7		"4D2"		"4"
	8	0			

6.9 The Scope of Variables

The scope of a variable is the part of the program where the variable can be referenced.

Scope of Variables

- **Scope:**
  - the part of the program where the variable can be referenced.
- **Local Variables:**
  - A variable created inside a function is referred to as a **local variable** .
  - Local variables can only be accessed inside a function.
  - The scope of a local variable starts from its creation and continues to the end of the function that contains the variable.
- **Global Variables:**
  - In Python, you can also use **global** variables .
  - They are created outside all functions and are accessible to all functions in their scope.

Example 1

Attempting to access the variable from **outside of the function** causes an error in line 9.

```
1  globalVar = 1
2  def f1():
3      localVar = 2
4      print(globalVar)
5      print(localVar)
6
7  f1()
8  print(globalVar)
9  print(localVar) # Out of scope, so this gives an error
```

Example 2

- Here a global variable x is created in line 1
- a local variable with the same name (x) is created in line 3.
  - From this point on, the global variable x is not accessible in the function.
- Outside the function, the global variable x is still accessible.

```
1  x = 1
2  def f1():
3      x = 2
4      print(x) # Displays 2
5
6  f1()
7  print(x) # Displays 1
```

Example 3

- Here the variable y is created if x > 0.
  - If you enter a positive value for x (line 1), the program runs fine.
  - But if you enter a nonpositive value, line 5 produces an error because y is not created.

```
1  x = eval(input("Enter a number: "))
2  if x > 0:
3      y = 4
4
5  print(y) # This gives an error if y is not created
```

Example 4

- Here the variable i is created in the loop.
- After the loop is finished, i is 4,
  - so line 5 displays 4.

```
1  sum = 0
2  for i in range(0, 5):
3      sum += i
4
5  print(i)
```

Example 5

- You can bind a local variable in the global scope.
- You can also create a variable in a function and use it outside the function.
- To do either, use a **global** statement

```
1  x = 1
2  def increase():
3      global x
4      x =  x + 1
5      print(x) # Displays 2
6
7  increase()
8  print(x) # Displays 2
```

Global Variables vs Global Constants

- Global variables can be used and modified across functions
- Although allowed in Python, **modifying global variables inside functions is not recommended**
- This practice can lead to:
  - Unexpected behavior
  - Hard-to-find bugs
  - Reduced program reliability

Global Variables vs Global Constants

- **Good Practice**
  - Avoid changing global variables inside functions
- **Recommended Approach**
  - Use **global constants** instead
  - Constants are shared across all functions in a module
  - Their values should **not be changed**

6.10 Default Arguments

Python allows you to define functions with default argument values. The default values are passed to the parameters when a function is invoked without the arguments.

Default Arguments

- Python allows you to define functions with default argument values.
- The default values are passed to the parameters when a function is invoked without the arguments.

DefaultArgumentDemo.py

```
1  def printArea(width = 1, height = 2):
2      area = width * height
3      print("width:", width, "\theight:", height, "\tarea:", area)
4
5  printArea() # Default arguments width = 1 and height = 2
6  printArea(4, 2.5) # Positional arguments width = 4 and height = 2.5
7  printArea(height = 5, width = 3) # Keyword arguments width
8  printArea(width = 1.2) # Default height = 2
9  printArea(height = 6.2) # Default width = 1
```

```
width: 1        height: 2       area: 2
width: 4        height: 2.5     area: 10.0
width: 3        height: 5       area: 15
width: 1.2      height: 2       area: 2.4
width: 1        height: 6.2     area: 6.2
```

Default Arguments – Important Rule

- A function can mix:
  - Parameters with default values
  - Parameters without default values
- **Rule:** Non-default parameters must come before default parameters
- Incorrect order can cause errors

Incorrect Example (Error)

```
def add(a=5, b):
    return a + b
```

```
SyntaxError: parameter without a default follows
parameter with a default
```

Correct Example

```
def add(a, b=5):
    return a + b

print(add(10))       # uses default b=5
print(add(10, 20))   # overrides default
value
```

Function Overloading in Python

- **No Function Overloading in Python**
  - Some languages allow multiple functions with the same name
  - Python does **NOT support function overloading**
  - Defining a function again replaces the previous one
- **Using Default Arguments Instead**
  - Default arguments allow one function to behave in multiple ways
  - This replaces the need for multiple functions with the same name
  - Makes code simpler and more flexible

6.11 Returning Multiple Values

The Python return *statement can return multiple values.*

Returning Multiple Values

- Python allows a function to return multiple values.
- Following code defines a function that takes two numbers and returns them in non descending order.

MultipleReturnValueDemo.py

```
1  def sort(number1, number2):
2      if number1 < number2:
3          return number1, number2
4      else:
5          return number2, number1
6
7  n1, n2 = sort(3, 2)
8  print("n1 is", n1)
9  print("n2 is", n2)
```

```
n1 is 2
n2 is 3
```

6.12 Case Study: Generating Random ASCII Characters

A character is coded using an integer. Generating a random character is to generate an integer.

Generating Random Characters

LISTING 6.11 RandomCharacter.py

```
1  from random import randint # import randint
2
3  # Generate a random character between ch1 and ch2
4  def getRandomCharacter(ch1, ch2):
5      return chr(randint(ord(ch1), ord(ch2)))
6
7  # Generate a random lowercase letter
8  def getRandomLowerCaseLetter():
9      return getRandomCharacter('a', 'z')
10
11 # Generate a random uppercase letter
12 def getRandomUpperCaseLetter():
13     return getRandomCharacter('A', 'Z')
14
15 # Generate a random digit character
16 def getRandomDigitCharacter():
17     return getRandomCharacter('0', '9')
18
19 # Generate a random character
20 def getRandomASCIICharacter():
21     return chr(randint(0, 127))
```

LISTING 6.12 TestRandomCharacter.py

```
1  import RandomCharacter
2
3  NUMBER_OF_CHARS = 175 # Number of characters to generate
4  CHARS_PER_LINE = 25 # Number of characters to display per line
5
6  # Print random characters between 'a' and 'z', 25 chars per line
7  for i in range(NUMBER_OF_CHARS):
8      print(RandomCharacter.getRandomLowerCaseLetter(), end = " ")
9      if (i + 1) % CHARS_PER_LINE == 0:
10         print()  # Jump to the new line
```

```
gmjsohezfkgtazqgmswfclrao
pnrunulnwmaztlfjedmpchcif
lalqdgivxkxpbzulrmqmbhikr
lbnrjlsopfxahssqhwuuljvbe
xbhdotzhpehbqmuwsfktwsoli
cbuwkzgxpmtzihgatdslvbwbz
bfesoklwbhnooygiigzdxuqni
```

6.13 Function Abstraction and Stepwise Refinement

Function abstraction is achieved by separating the use of a function from its implementation.

Function Abstraction

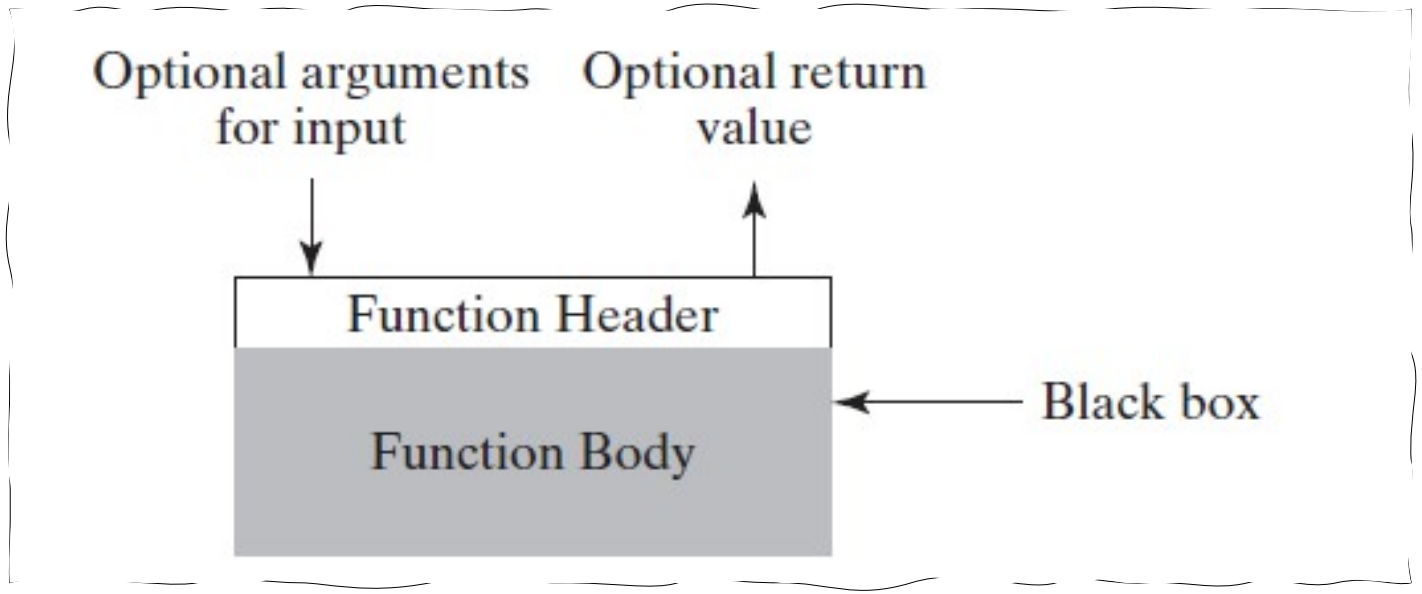
- Function abstraction is achieved by separating the use of a function from its implementation.
- **Key Idea**
  - The key to developing software is the concept of abstraction.
  - Abstraction allows focusing on *what a function does* rather than *how it is implemented*.

Function Abstraction

- **Encapsulation / Information Hiding**
  - Implementation details are hidden inside the function.
  - This concept is called:
    - Information hiding
    - Encapsulation
- **Benefits**
  - Changes in implementation do NOT affect the client program.
  - Only the function header must remain unchanged.
  - Improves modularity and maintainability.

Function Abstraction

- You can think of the function body as a black box that contains the detailed implementation for the function.



Benefits of Functions

- Write a function once and reuse it anywhere.
- Information hiding.
  - Hide the implementation from the user.
- Reduce complexity.

Stepwise Refinement

- The concept of function abstraction can be applied to the process of developing programs.
- When writing a large program, you can use the “divide and conquer” strategy, also known as stepwise refinement , to decompose it into sub problems.
- The sub problems can be further decomposed into smaller, more manageable problems.

To function or not to function...

Organize your code into “paragraphs” - capture a complete thought and “name it”

Don’t repeat yourself - make it work once and then reuse it

If something gets too long or complex, break it up into logical chunks and put those chunks in functions

Make a library of common stuff that you do over and over - perhaps share this with your friends...

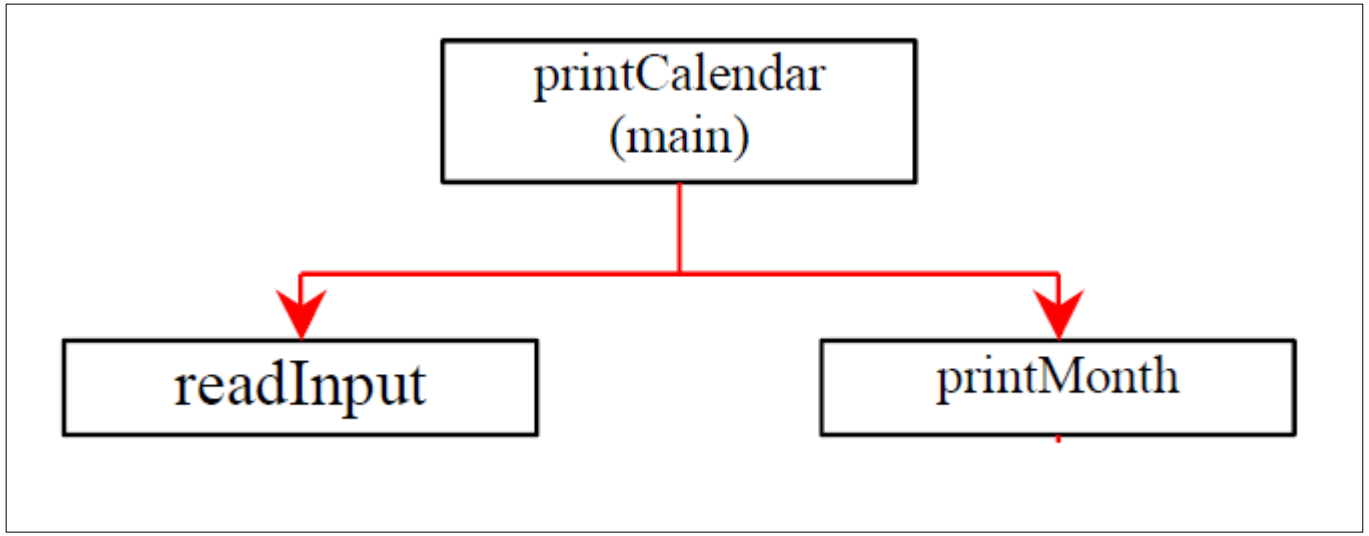
PrintCalendar Case Study

```
Enter full year (e.g., 2001): 2011  ↵Enter
Enter month as number between 1 and 12: 9  ↵Enter
         September 2011
-----------------------------
 Sun  Mon  Tue  Wed  Thu  Fri  Sat
                    1    2    3
   4    5    6    7    8    9   10
  11   12   13   14   15   16   17
  18   19   20   21   22   23   24
  25   26   27   28   29   30
```

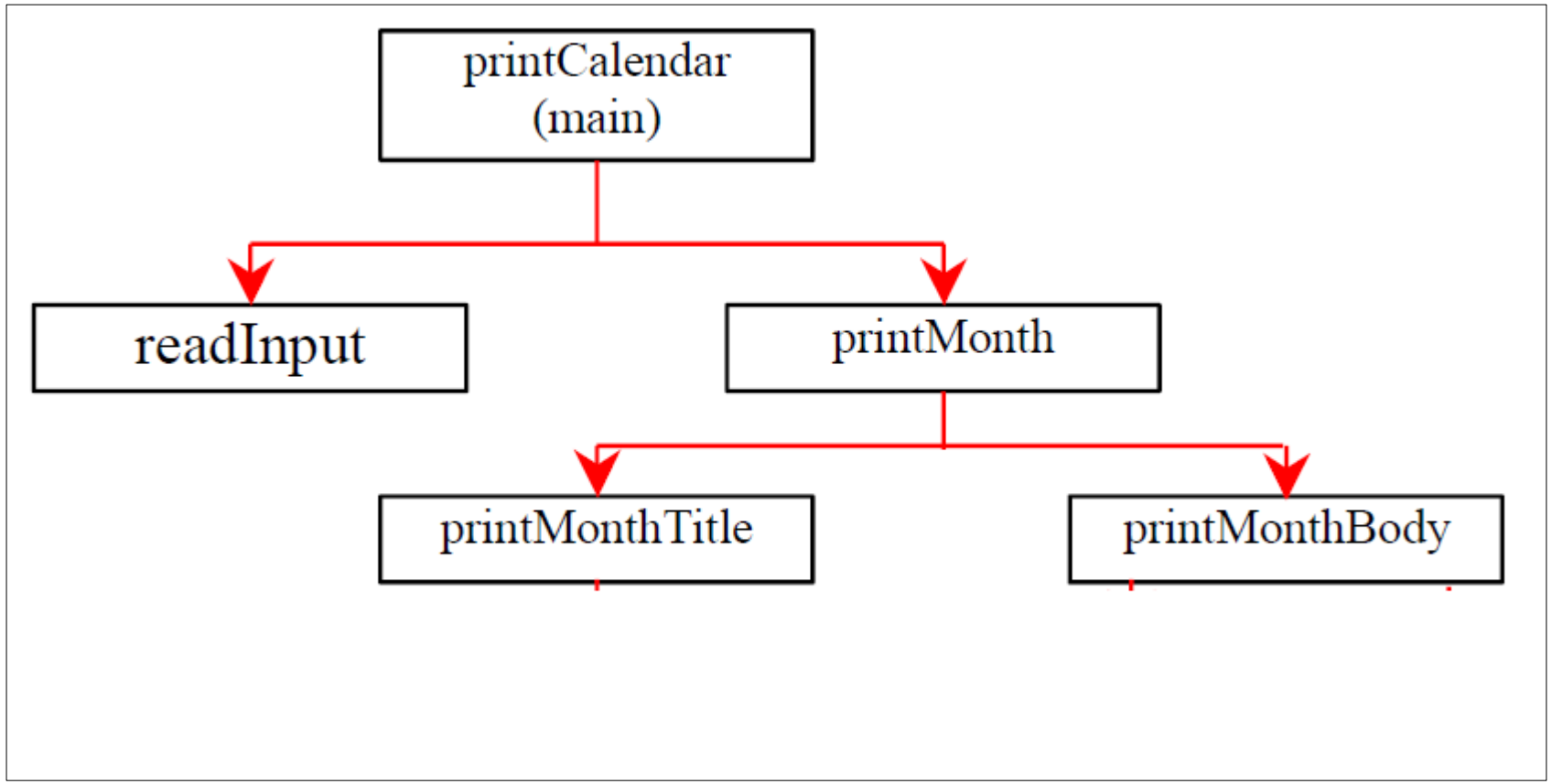
Top-Down Design

- **Top-Down Design (Introduction)**
  - How do we start writing a program?
  - Beginners often start coding immediately
  - This leads to focusing too early on details
  - Early focus on details can block problem-solving flow
- **Better Approach**
  - Use **Top-Down Design**
  - Start by understanding the problem at a high level
  - Delay implementation details until later
  - Use function abstraction to manage complexity

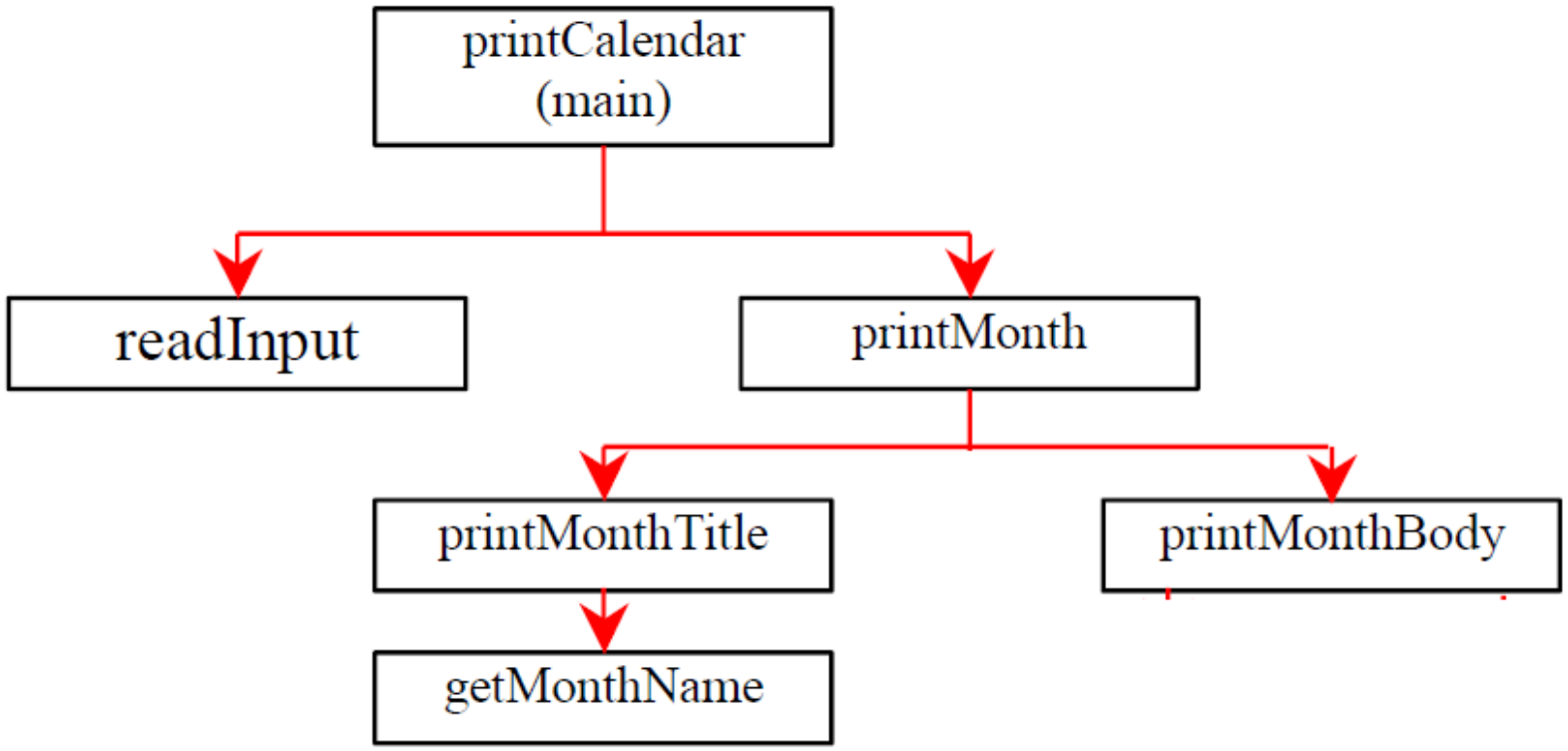
Design Diagram



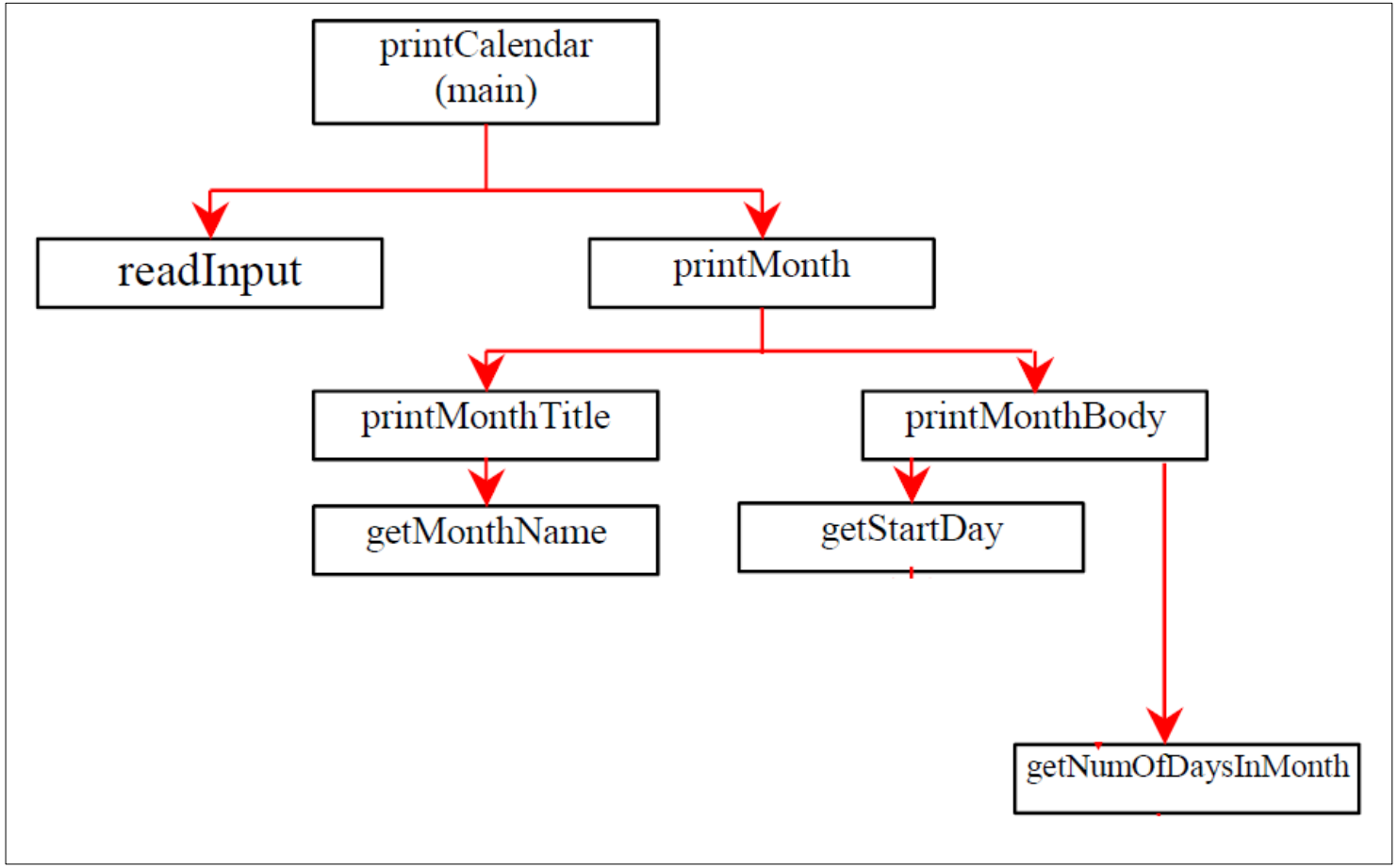
Design Diagram



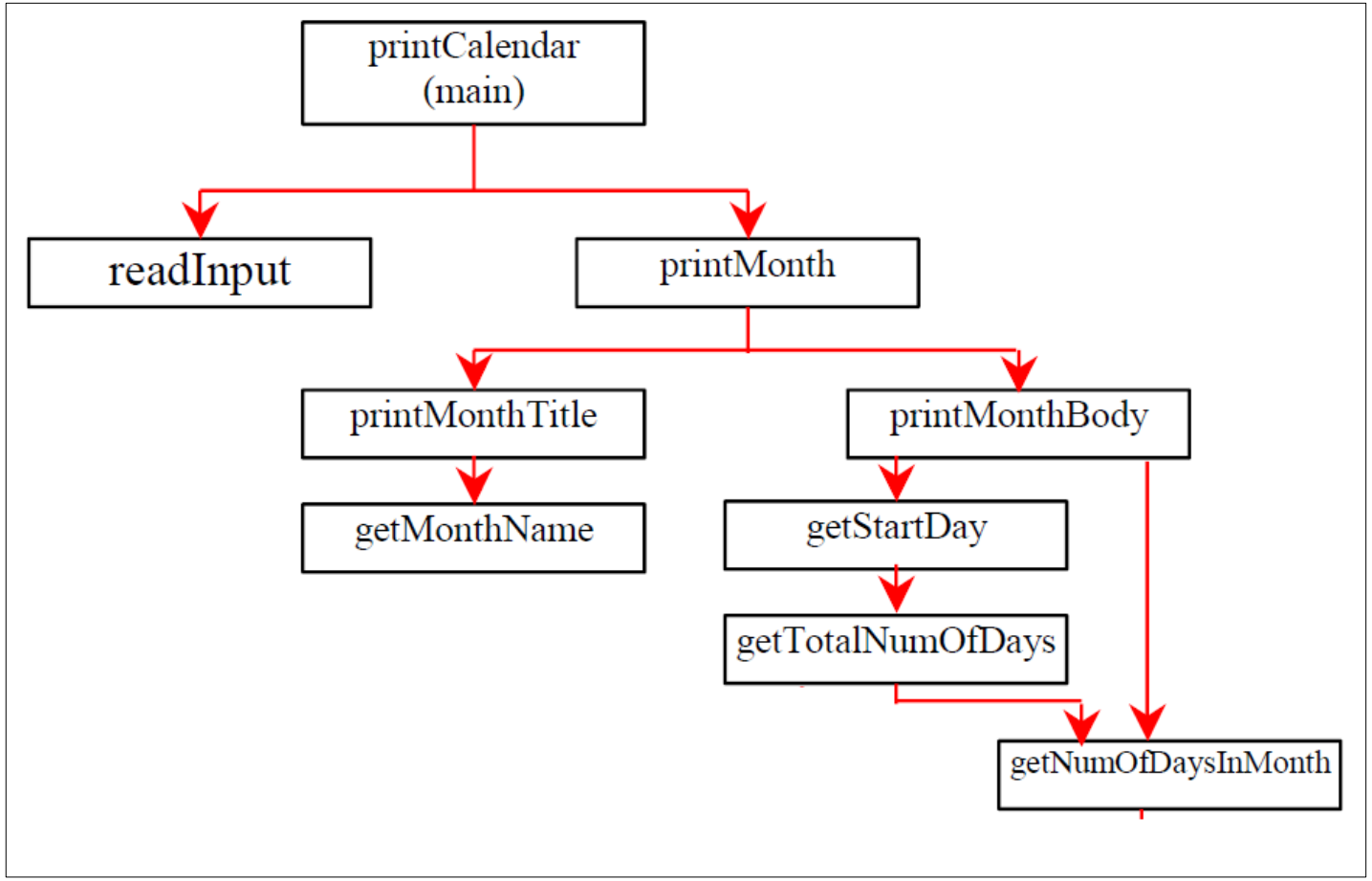
Design Diagram



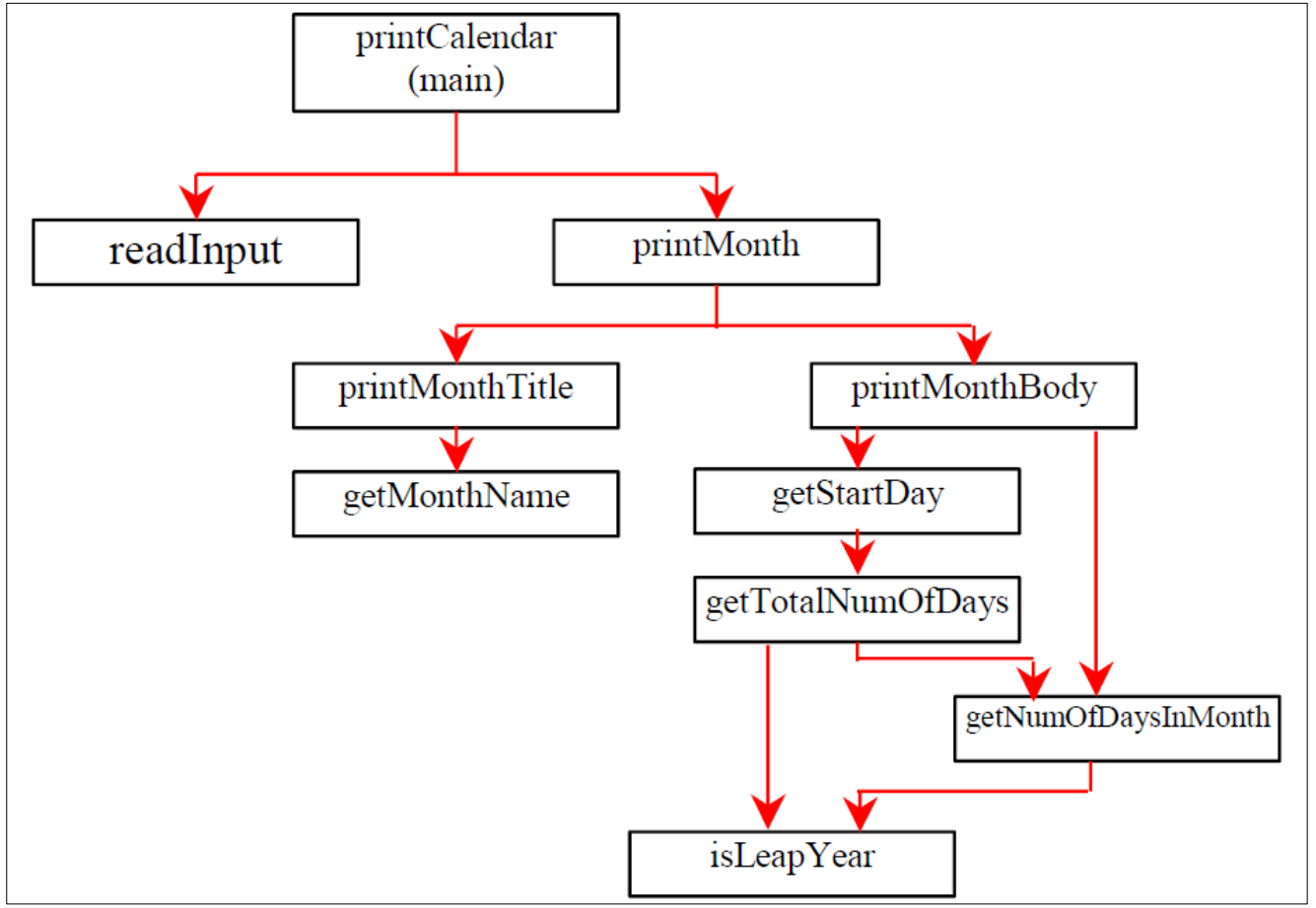
Design Diagram



Design Diagram



Design Diagram



Implementation: Top - Down

- Top down approach is to implement one function in the
- structure chart at a time from the top to the bottom.
- Stubs can be used for the functions waiting to be implemented.
- A stub is a simple but incomplete version of a function.
- The use of stubs enables you to test invoking the function from a caller.
- Implement the main function first and then use a stub for the printMonth function.
  - For example, let printMonth display the year and the month in the stub.
- Thus, your program may begin like this:

Implementation: Top - Down

```
# A stub for printMonth may look like this
def printMonth(year, month):
    print(year, month)

# A stub for printMonthTitle may look like this
def printMonthTitle(year, month):
    print("printMonthTitle")

# A stub for getMonthBody may look like this
def getMonthBody(year, month):
    print("getMonthBody")

# A stub for getMonthName may look like this
def getMonthName(month):
    print("getMonthName")
```

PrintCalendar.py

```
# A stub for getStartDay may look like this
def getStartDay(year, month):
    print("getStartDay")

# A stub for getTotalNumberOfDays may look like this
def getTotalNumberOfDays(year, month):
    print("getTotalNumberOfDays")

# A stub for getNumberOfDaysInMonth may look like this
def getNumberOfDaysInMonth(year, month):
    print("getNumberOfDaysInMonth")

# A stub for isLeapYear may look like this
def isLeapYear(year):
    print("isLeapYear")

def main():
    # Prompt the user to enter year and month
    year = eval(input("Enter full year (e.g., 2001): "))
    month = eval(input((
       "Enter month as number between 1 and 12: ")))

    # Print calendar for the month of the year
    printMonth(year, month)

main() # Call the main function
```

Implementation: Bottom-Up

- Bottom up approach is to implement one function in the structure chart at a time from the bottom to the top.
- For each function implemented, write a test program to test it.
- Both top down and bottom up functions are fine.
- Both approaches implement the functions incrementally and
- help to isolate programming errors and makes debugging easy.
- Sometimes, they can be used together.

End of Chapter 06